

Algoritmi Paralleli e Distribuiti

simi

February 2024

Contents

1	Introduzione	3
1.1	Algoritmi Paralleli	4
1.2	Algoritmi Distribuiti	5
1.3	Teoria e realtà	5
1.4	Tempo	6
1.4.1	Efficienza	7
1.5	Problematiche algoritmi paralleli	8
1.5.1	Sintesi	8
1.5.2	Valutazione delle prestazioni	8
1.5.3	Universalità	8
1.6	Tipi di architetture parallele	8
1.6.1	Memoria distribuita	9
2	PRAM	10
2.1	Come si programma una PRAM	11
2.2	Risorse di calcolo	12
2.3	Efficienza	13
2.3.1	Speed-up	13
2.3.2	Efficienza	14
2.4	Sommatoria	14
2.4.1	Valutazione	18
2.5	OP Iterata	20
2.5.1	CRCW - PRAM	20
2.6	Prodotto interno di vettori	21
2.6.1	Prodotto matrice vettore	21
2.6.2	Prodotto MATRICE MATRICE	22
2.6.3	Potenza di matrice	22
2.6.4	Somme Prefisse	22
2.6.5	Valutazione dei polinomi	27
2.7	Ricerca di un elemento	29
2.7.1	Varianti per problemi legati	30
2.8	Ordinamento	31
2.8.1	Bit-Sort	32
2.8.2	Osservazioni	39
2.9	Tecnica del ciclo Euleriano	39
2.9.1	Primo passo	40

2.9.2	Secondo passo	40
2.9.3	Terzo passo	41
2.9.4	Attraversamento dell'albero in preordine	42
2.9.5	Calcolo della profondità dei nodi	42
3	Architetture Parallele a Memoria Distribuita	44
3.1	Comparatori	46
3.2	Array Lineari	48
3.2.1	Shuffle	49
3.2.2	MAX	49
3.2.3	Ordinamento	51
3.3	Mesh	53
3.3.1	MAX	54
3.3.2	Ordinamento	55
4	Ambiente di calcolo distribuito	59
4.1	BCAST	63
4.2	Wake-Up	65
4.3	Traversal	66
4.4	Spanning Tree	72
4.5	Election	75
4.6	Routing	79
4.6.1	Shortest Path	79

Chapter 1

Introduzione

Un' algoritmo è una sequenza finita di istruzioni eseguibili e non ambigue che portano ad un risultato, ovvero ad ogni istanza di input termina. L'esecutore termina ad ogni input. Siccome si parla di sequenza di istruzioni eseguite una dopo l'altra è sottinteso che ci sia un unico esecutore, perché quando fu coniato questo termine gli algoritmi erano sequenziali, ed erano gli unici esistenti all'epoca. Noi invece parliamo di paralleli e distribuiti, questi termini danno un'altra luce al concetto di algoritmi, pensiamo ad algoritmi che devono essere eseguiti da più esecutori. In questo caso dovremmo parlare di insiemi di istruzioni raggruppati in passi, avremo passi paralleli in cui è possibile avere un set di istruzioni, uno per ogni esecutori. La stessa cosa per i distribuiti, avremo più esecutori, non si può parlare di sequenza ma di passi dell'algoritmo distribuito, dove avremo più istruzioni eseguite contemporaneamente. La differenza tra paralleli e distribuiti sta nel fatto che gli esecutori, che lavorano insieme, possono essere sincronizzati tra di loro o possono essere asincroni, se sincronizzati abbiamo algoritmi paralleli, al contrario parliamo di algoritmi distribuiti.

Per analizzare gli algoritmi paralleli e distribuiti gli avviciniamo sotto tra differenti aspetti, esattamente come venivano approcciati gli algoritmi sequenziali:

- progettare: significa pensare ad un problema e a volte può essere risolto in maniera banale o furba. Se risolto in maniera furba è perché vengono usate tecniche note, come divide et impera, programmazione dinamica o tecnica greedy. Nella progettazione degli algoritmi entrano in gioco tecniche.
- valutare le prestazioni: nel caso degli algoritmi sequenziali si misuravano utilizzando la risorsa tempo e spazio di memoria
- codificare: implementazione degli algoritmi con un opportuno linguaggio di programmazione

Le stesse problematiche le affrontiamo per il caso degli algoritmi paralleli e distribuiti. L'unica cosa che sappiamo al momento è che questi algoritmi sono rivolti ad un set di esecutori, però gli algoritmi paralleli e distribuiti sono due casi completamente diversi:

- Algoritmi paralleli: gli esecutori sono sincroni, abbiamo un unico clock
- Algoritmi distribuiti: ogni esecutore sta in un mondo a parte, sono asincroni, ognuno ha il suo clock

1.1 Algoritmi Paralleli

Gli esecutori condividono un clock centrale ed eseguono insieme le istruzioni, vanno all'unisono. Siccome devono lavorare insieme per portare a termine un algoritmo devono a volte comunicare tra di loro, ad esempio dei risultati parziali di qualche operazione che hanno compiuto, per comunicare possono usare delle risorse, come la memoria (architetture a memoria condivisa). Oppure hanno tra di loro dei collegamenti full duplex (permette la comunicazione in entrambe le direzioni) e si spediscono i risultati parziali ottenuti (architettura a memoria distribuita).

Vediamo un esempio di algoritmo parallelo su un'architettura a memoria distribuita, vogliamo sommare 4 numeri, ci sono 4 processori e i numeri da sommare sono A, B, C,

D. Siccome non condividono la memoria ogni processore deve tenere un dato dei quattro numeri da sommare, inoltre per comunicare abbiamo delle connessioni. Quello che succede è che alcuni processori mandano il proprio numero agli altri utilizzando l'istruzione send, i processori che ricevono il dato fanno la somma con il proprio dato. Otteniamo dei risultati parziali che vengono combinati spedendoli. Le istruzioni non sono più in sequenza, ma possono avvenire in parallelo (vengono scritte sulla stessa riga), il programma non è più una sequenza di istruzioni, ma in un unico clock possono essere eseguite più operazioni. In genere è il processore con l'indice più alto a dare il risultato.

1.2 Algoritmi Distribuiti

É un pool di esecutori dove ognuno ha un proprio clock centrale e quindi ha i suoi tempi che non combaciano con gli altri. I vari processori sono collegati per comunicare, ma sicuramente non possiamo assumere che abbiamo una memoria in comune. Non possiamo assumere alcun tipo di sincronizzazione e per collaborare devono scambiarsi dei messaggi di sincronizzazione.

Prendiamo il problema di prima, sommare 4 numeri. Nel momento in cui si eseguono in parallelo le istruzioni di send non è detto che queste vengano eseguite contemporaneamente, ma potrebbero essere eseguite in tempi diversi. Potrebbero anche essere eseguite allo stesso momento ma potrebbero finire in tempi diversi. Potrebbero anche esserci tempi di trasmissione dei messaggi diversi. Per mettere d'accordo i vari processori potrebbero servire segnali di coordinamento. In questo caso è la comunicazione che assume un ruolo più rilevante rispetto al tempo di esecuzione delle operazioni.

1.3 Teoria e realtà

La motivazione per lo studio di questi algoritmi arriva dalla realtà, dal fatto che dalla seconda metà degli anni '60 si sono rese disponibili queste architetture e da quando si sono rese disponibili si è sviluppata la teoria. Esempi di architetture parallele sono:

- Supercomputer: sono un cluster di processori ad elevate prestazioni di calcolo (CDC, CRAY). Questi vengono utilizzati per simulare situazioni reali complesse (fenomeni fisici).
- GPU: sarebbero dedicate alla grafica, ma vengono utilizzate in senso più generale (AI)
- Multi-core processor: processori con più core, ottimizzano l'assorbimento di energia e relativi problemi di raffreddamento
- Circuiti integrati: componenti di sistemi di calcolo insieme di gate opportunamente connessi

Esempi di architetture distribuite:

- reti di calcolatori: connessi ad Internet, la rete permette di coprire grandi distanze quindi troveremo i dispositivi di calcolo lontani tra di loro, per questo ci sarà bisogno di un protocollo di comunicazione condiviso, TCP/IP

- reti mobili: la topologia delle connessioni cambia durante l'esecuzione di un algoritmo
- reti di sensori: dispositivi con limitate capacità, tenuti in stand by per la maggior parte del tempo. Si attivano attraverso un messaggio di wake up. Servono per monitorare delle realtà, come ad esempio la temperatura di un edificio.

Come dicevamo nel caso parallelo il fattore rilevante è il tempo, per l'esecuzione delle operazioni. Mentre nel distribuito il fattore rilevante è il coordinamento, quindi più che considerare il tempo di esecuzione delle singole istruzioni considereremo il numero di messaggi che si scambiano e ci danno il tipo di congestione che l'algoritmo produce quando viene eseguito, meno messaggi si scambiano le entità e più veloce risulta essere l'algoritmo distribuito.

Per ogni paradigma dovremo descrivere questo modello con un modello teorico, insieme a come si presenta questo modello di calcolo descriveremo anche le operazioni elementari che il modello di calcolo offre, perchè poi su queste operazioni andremo a valutare gli algoritmi. Vedremo poi problemi semplici che ci faranno da guida per descrivere le tecniche che si possono usare per questi paradigmi di calcolo.

Il caso di algoritmi paralleli si divide in due:

- PRAM: set di processori con memoria condivisa. Questa memoria condivisa fa sì che la comunicazione tra i processori è immediata. Su questo modello progetteremo degli algoritmi e li valuteremo utilizzando le risorse di calcolo, ovvero il tempo e il numero di processori. Andremo a risolvere i problemi di sommatoria, somme prefisse (data una sommatoria individuare i risultati parziali dei prefissi delle somme) e di ordinamento
- Modello a memoria distribuita: sono processori che non condividono una memoria ma una rete di interconnessione, collegati da fili full duplex. Abbiamo il modello degli array lineari (una linea di processori collegati in linea) e il modello a mesh (sono dei processori strutturati a forma di quadrato, con connessioni sia per riga che per colonna). In questo caso le risorse di calcolo sono il tempo e l'HW, nel tempo contiamo anche le operazioni di scambio di informazioni in quanto la comunicazione non è immediata. Affronteremo problemi di shuffle (prende un array, lo divide a metà e intercala la prima metà con la seconda), di massimo e di ordinamento

Il secondo paradigma è quello distribuito, daremo una descrizione teorica dell'architettura distribuita. Come risorse di calcolo useremo il tempo, ma invece dell'HW calcoleremo il costo della comunicazione, ovvero il numero di messaggi che definisce la congestione del sistema nel momento in cui manderemo in esecuzione l'algoritmo. Inizialmente affronteremo problemi legati ad una sincronizzazione dei processori che partecipano all'architettura che saranno il broadcast, il wake up e il traversal. Successivamente affronteremo problemi rivolti all'utente finale come spanning tree, election e il routing.

1.4 Tempo

In tutti e tre i modelli che sono stati presentati la risorsa che compare sempre è il tempo. Quindi il tempo risulta essere fondamentale. Ripercorriamo ciò che è stato fatto per gli

algoritmi sequenziali. Come viene misurato il tempo? viene misurato come la quantità di operazioni elementari richieste per risolvere il problema, nel caso sequenziale si utilizzava una funzione $t(n) = \max\{T(x) | x \in \Sigma^n\}$, dove entra in gioco la funzione $T(x)$ che rappresenta il numero di operazioni elementare su x , dove x è l'istanza. Quindi per sapere il tempo abbiamo una funzione in n , dove n è la lunghezza dell'input, quindi troveremo il caso peggiore, ovvero il numero massimo di operazioni necessarie per risolvere l'istanza.

Spesso non saremo interessati ad una valutazione precisa di $t(n)$, ma al suo tasso di crescita (o ordine di grandezza). Per fare questo useremo le funzioni asintotiche: O, Ω, Θ . Siano f, g due funzioni da N in N sui naturale diremo che:

- $f(n) = O(g(n))$ se e solo se esiste una costante $c > 0$ e $n_0 \in N$ tale che $f(n)$ Limitata superiormente da $g(n)$
- Limitata inferiormente fa $g(n)$
- Sta tra le due

Abbiamo anche delle avvertenze quando valutiamo $t(n)$, la valutazione del tempo dipende dal modello di calcolo, inoltre bisogna stabilire se utilizzeremo un certo criterio per valutare il tempo se utilizzeremo il costo uniforme o logaritmico. Ad esempio per risolvere il problema di ricerca di parole palindrome in una macchina ram avremo un costo di $O(n)$, mentre su una macchina di touring avremo un costo di $O(n^2)$.

Bisogna fare anche attenzione alla dimensione dei dati in gioco:

- Se uso un costo uniforme le operazioni elementari richiedono una unità di tempo (il tempo richiesto è uno)
- Se ho un costo logaritmico ogni operazione elementare ha un costo che dipende dal numero di bit degli operandi (la somma di due numeri binari di n bit è n)

1.4.1 Efficienza

La funzione tempo deve dirci se gli algoritmi sono efficienti. Nel caso sequenziale si era definita l'efficienza come: un problema è risolto efficientemente in tempo se e solo se è risolta da una macchina di Turing (input sequenziale con una testina e un controllo a stati, ad ogni istante si legge l'elemento in input e sullo stato) in tempo polinomiale. Sono state definite le classi:

- P classe di problemi di decisione risolti efficientemente in tempo
- FO: classe di problemi generali risolti efficienti in tempo

Risulta da quanto abbiamo detto che tempo polinomiale significa efficienza, questo perchè una soluzione efficiente risulta essere robusta rispetto a svariati modelli di calcolo. Ci sono diversi problemi che stanno in P e FP, come ordinamento, prodotto di matrici, ricerca di cammino minimo (problemi di ottimizzazione). Ci sono altri problemi che ancora non sono stati classificati in P e FP, ad esempio problemi di soddisfacibilità di formule e fattorizzazione di interi (problema utile per algoritmi di crittografia), questi problemi stanno in NP, possono essere risolti in tempo polinomiale ma su una macchine di Turing non deterministica.

1.5 Problematiche algoritmi paralleli

- Sintesi: costruzione
- Valutazione
- Universalità

1.5.1 Sintesi

Come possiamo costruire dei buoni algoritmi paralleli? ci possiamo ispirare a buoni algoritmi sequenziali e distribuire le operazioni a più processori ed ottenere così algoritmi paralleli? Ispirarsi è possibile, ma riciclare l'intero algoritmo non si può riciclare quasi mai.

Ad esempio vediamo come sommare due numeri binari, l'algoritmo sequenziale ottimo è quello che somma i bit per colonna. Potremmo voler prendere le operazioni elementari di questo algoritmo, distribuirle tra processori e trovare un ottimo algoritmo parallelo, ad esempio ogni processore si può occupare di una colonna, un problema che non abbiamo considerato è il riporto quindi se un processore deve aspettare un riporto per fare la sua operazione aspetta tanto quanto un singolo processore che deve fare tutta la somma. Torneremmo ad avere un processore sequenziale con in più uno spreco di HW.

Vediamo un altro esempio di ordinamento, un ottimo algoritmo di ordinamento è il merge sort, non è possibile parallelizzare questo algoritmo prettamente sequenziale (potremmo pensare di avere un set di processori che prendono pezzi di array da ordinare, ma poi si arriva al merge che ci riporta ad un algoritmo sequenziale).

Per la sintesi di algoritmi paralleli servono idee nuove.

1.5.2 Valutazione delle prestazioni

Dobbiamo considerare tempo e numero di processori. Ma cosa è efficiente nel caso parallelo? in questo caso il tempo non può essere polinomiale perchè usiamo HW in più, se ottenessimo la stessa efficienza degli algoritmi sequenziali ci terremmo quelli.

Dovremo definire un parametro E che tiene conto sia del tempo, sia del numero di processori e deve dire se il gioco vale la candela, nel senso che se devo avere un'esplosione esponenziale dei processori potrebbe non valerne la pena.

1.5.3 Universalità

Non ci occuperemo di questa problematica che è uno studio teorico di cosa è parallelo. Significa descrivere i problemi che possono essere risolti in tempo parallelo efficiente.

Abbiamo la classe NC di problemi, ovvero problemi risolti da algoritmi paralleli veloci con tempo polilogaritmico e HW polinomiale.

1.6 Tipi di architetture parallele

Abbiamo visto che esistono due architetture:

- A memoria condivisa: c'è una memoria centrale su cui si affacciano i processori. Tutti i processori sono collegati ad una memoria che è condivisa tra di loro. I processori hanno anche la memoria privata.

Una proprietà non visibile è che i processori lavorano all'unisono, esiste un solo clock centrale. Quando i processori vogliono comunicare lo fanno usando la memoria centrale, questa comunicazione avviene in modo costante. Se p_j vuole comunicare scrive il dato sulla memoria centrale e p_x legge il dato, questa è la situazione dei multiprocessori attuali. Questa architettura permette una forte parallelizzazione. Meglio di così non possiamo parallelizzare gli algoritmi. Il motivo è dato dalla comunicazione, perchè la comunicazione richiede tempo costante, non abbiamo quindi problemi di comunicazione. La comunicazione avviene in maniera sincrona, non si spreca tempo per la comunicazione.

1.6.1 Memoria distribuita

I processori sono sincroni ma non hanno una risorsa condivisa, al posto di una memoria centrale c'è una rete di interconnessione che lega i vari processori. In genere la rete di interconnessione non mi dà un grafo completo. Naturalmente questi processori hanno una memoria privata per poter lavorare. Comunque questi processori condividono un unico clock centrale, questo è il caso dei super computer.

La comunicazione non è più costante ma dipende dalla distanza tra i processori. La comunicazione è diretta nel caso in cui c'è un link diretto tra i due processori, quando non c'è il link la comunicazione dipende dal cammino che deve compiere il messaggio. La distanza quindi è il numero di processori che sono presenti sul collegamento tra i due (devono sapere la topologia).

Come possono essere queste reti:

- Array lineare: array connessi uno in fila all'altro
- Mesh: quadrato di processori, connessi per riga e per colonna
- Ipercubo: cubo di processori

Chapter 2

PRAM

Ricordiamo che il modello ha più processori che si affacciano su una memoria condivisa fatta da una sequenza di registri $M[0] \dots M[n]$, è un modello abbastanza attuale se il numero di processori è contenuto. Noi quando ci lavoreremo avremo un numero di processori che varia in base alla dimensione dell'input.

Ogni singolo processore p_i è una RAM (macchina sequenziale) costituita da una unità di calcolo (ALU) e i suoi registri personali che indicheremo con $R[0] \dots R[n]$. Quando l'unità di calcolo deve compiere delle operazioni deve avere i dati nella memoria personale, noi in realtà dato che usiamo tempo costante faremo somme in M .

Le istruzioni che può compiere ogni processore sono:

- Operazioni aritmetiche e logiche
- Istruzioni per elaborare dati nella ram, istruzioni da e per la memoria centrale:
 - STORE: memorizzare un dato nella RAM
 - LOAD: prendere un dato dalla RAM

Vediamo un esempio di comunicazione, P_j vuole comunicare con P_i il contenuto di $R[t]$:

```

1  $P_j$ : STORE  $R[t]$   $M[x]$ 
2  $P_i$ : LOAD  $R[s]$   $M[x]$ 

```

Notiamo che dobbiamo passare dalla stessa cella della memoria centrale. La comunicazione avviene in tempo $O(1)$. Viene detto adesso ma non lo useremo per non appesantire gli algoritmi, lavoreremo sui registri M .

2.1 Come si programma una PRAM

Abbiamo un unico programma per tutti i processori (non interessa dove sta), tutti i processori hanno accesso a questo programma che contiene tutte le istruzioni, ad ogni ciclo di clock (Centrale) ogni processore esegue la stessa istruzione, anche se in realtà qui non può essere la stessa istruzione identica, ma le istruzioni si differenziano in funzione dell'indice del processore.

Troviamo la forma dell'istruzione, chiamato anche passo parallelo:

```

1   for all  $i$  in  $I$  par do:
2       istruzione  $i$ 

```

Si differenzia da un for sequenziale perchè si usa par. I processori con indice in I eseguono l'istruzione i . Mentre i processori con indice NON in I eseguono l'istruzione nulla (non fanno niente).

Ma cos'è l'istruzione i ? Nella PRAM possiamo avere due architetture diverse:

- SIMD: single instruction multiple data. L'istruzione è uguale per tutti i processori ma avviene su dati diversi.
- MIMD: multiple instruction multiple data. C'è un modo nell'istruzione i di far capire al processore che istruzione eseguire, sempre su dati diversi.

Possiamo avere diverse architetture in base alla capacità che ci offrono per accedere alla memoria centrale:

- EREW: no lettura simultanea e no scrittura simultanea
- CREW: sì lettura simultanea ma no scrittura simultanea
- CRCW: su scrittura e lettura simultanea

dove:

- E = exclusive. Due processori allo stesso tempo NON possono accedere alla stessa cella di memoria
- C = concurrent
- R = read
- W = write

Abbiamo politiche diverse quando due processori possono scrivere sulla stessa cella simultaneamente:

- common: i processori possono scrivere solo lo stesso dato, pena l'arresto del sistema
- random: si sceglie un processore a caso
- max/min: viene il processore con il dato max/min
- priority: vince il processore con priorità max

Osserviamo che se scriviamo un algoritmo per EREW gira bene anche su architetture CREW e un algoritmo di tipo CREW funziona bene su CRCW. Ma vale anche il viceversa. L'architettura più facile da costruire è di tipo EREW.

2.2 Risorse di calcolo

Nel caso sequenziale avevamo il tempo sequenziale $t(n)$ e le risorse di calcolo $s(n)$. Invece nel caso parallelo abbiamo il numero di processori $p(n)$ che non è fissato ma è in funzione di n e il tempo parallelo che è una funzione $T(n, p(n))$ in funzione dell'istanza dell'input e del numero di processori.

- $P(n)$: numero di processori richiesti su input di lunghezza n (caso peggiore)
- $T(n, p(n))$: tempo richiesto da un input di lunghezza n e $p(n)$ processori (caso peggiore). L'idea è che aumentando il numero di processori si migliora il tempo, non è sempre vero, si arriva ad una soglia.

Se a $p(n)$ gli mettiamo uno abbiamo il tempo sequenziale: $T(n, 1)$.

Vediamo un esempio più preciso di valutazione di $T(n)$. Abbiamo la batteria di processori e mettiamo in colonna le istruzioni eseguite dai processori, ogni riga mi dà il passo parallelo, il numero di passi paralleli in genere dipende da n . Ogni processore ad ogni passo ha un costo, il pedice indica il passo e l'apice indica il processore. Per calcolare il tempo parallelo di tutti i t ad ogni passo devo prendere il massimo, perchè comunque devo aspettare l'istruzione più lenta e lo chiamiamo:

$$t_i(n) = \max\{t_i^{(i)}(n) | 1 \leq i \leq p(n)\}$$

allora il tempo parallelo è la somma del tempo richiesto all' i -esimo passo per i che va da 1 a kn :

$$T(n, p(n)) = \sum_{i=1}^{k(n)} t_i(n)$$

Il tempo parallelo ovviamente dipende dai passi, dalla dimensione dell'input perchè ogni tempo i -esimo è un tempo sequenziale che dipende da n ed infine dipende da $p(n)$.

2.3 Efficienza

Gli algoritmi che dobbiamo progettare devono migliorare le prestazioni degli algoritmi sequenziali. Quindi calcoliamo la funzione $T(n, p(n))$ con la funzione $T(n, 1)$, ovvero il tempo del tempo sequenziale. Ci sono due possibilità:

- Il tempo parallelo è simile a quello sequenziale: $T(n, p(n)) = \Theta(T(n, 1))$
- Il tempo parallelo è un o-piccolo del tempo sequenziale, cresce più lentamente rispetto al tempo sequenziale: $T(n, p(n)) = o(T(n, 1))$

Il primo caso è quello che vogliamo evitare, invece vogliamo che il tempo parallelo sia un o-piccolo di quello sequenziale.

2.3.1 Speed-up

Lo usiamo per definire l'efficienza, è il rapporto tra il tempo sequenziale diviso il tempo parallelo.

$$S(n, p(n)) = \frac{T(n, 1)}{T(n, p(n))}$$

Se ci dà un numero significa che il tempo parallelo è n volte più veloce di quello sequenziale, ma così ricado nel caso in cui il tempo parallelo è un θ di quello sequenziale. Noi quindi vorremo che lo speed up sia infinito, in questo caso il tempo parallelo sarebbe un o-piccolo di quello sequenziale.

Questo parametro non ci dice nulla sul numero di processori, non va bene perchè non vogliamo usare un numero di processori infinito.

2.3.2 Efficienza

Per valutare l'algoritmo parallelo tenendo conto anche dei processori definiamo l'efficienza, che è il rapporto tra il tempo sequenziale diviso il tempo sequenziale per il numero di processori. Intendiamo il tempo migliore sequenziale:

$$E(n, p(n)) = \frac{T(n, 1)}{p(n) \cdot T(n, p(n))}$$

Vedremo che il parametro E è compreso tra 0 e 1. Se il parametro efficienza tende a 0 dobbiamo convincerci che l'algoritmo non è così buono perché stiamo usando troppi processori che magari vengono inutilizzati per la maggioranza del tempo.

Vediamo che l'efficienza è minore uguale a 1. Presentiamo il solito schema con la batteria di processori e i passi paralleli. Possiamo creare da questo algoritmo parallelo un algoritmo sequenziale facendo eseguire ad un processore tutti i passi in sequenza. Chiamiamo il tempo di questo algoritmo sequenziale T tilde $\tilde{T}(n, 1)$. Abbiamo dimostrato che l'efficienza è minore uguale a 1.

Se l'efficienza è E non va bene, perché quello che abbassa la funzione è il numero di processori, invece quando avremo una costante k maggiore di 0 siamo contenti e possiamo dire che l'algoritmo è un buon algoritmo parallelo.

Otteniamo una formula che dimostra che diminuendo i processori aumentiamo il parametro di efficienza, così facendo scopriamo che l'efficienza di qualsiasi algoritmo parallelo è limitata superiormente a 1.

2.4 Sommatoria

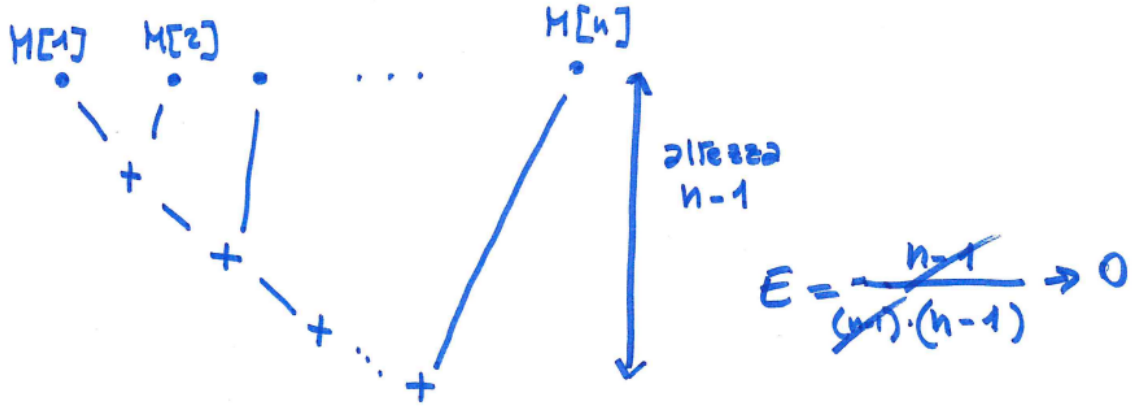
Iniziamo ad affrontare un primo problema di cui vogliamo trovare un algoritmo parallelo. Definiamolo bene, abbiamo n valori in input e supponiamo che siano in memoria centrale, ovvero nelle celle $M[1], \dots, M[n]$ e desideriamo in output la somma di questi valori che scriveremo in una cella della memoria centrale $M[n] = \sum_{i=1}^n M[i]$. C'è un buon algoritmo sequenziale che lo risolve, somma tutti i valori in una cella che usa come accumulatore:

```
1   for i = 1 to n-1 do
2       M[n] = M[n] + M[i]
```

In questo algoritmo ho $n-1$ somme, il tempo è $T(n, 1) = n - 1$.

Come si può parallelizzare? invece che usare un processore che fa le somme uno dietro l'altro si potrebbe usare un processore per ogni somma. Se facciamo così potremmo pensare di scomporre l'algoritmo sequenziale come un'albero, ogni riga ci dà un passo parallelo. Otteniamo così un tempo uguale a n e un'efficienza che tende a 0. Siamo nel caso in cui il tempo parallelo è uguale a quello sequenziale, non abbiamo progettato un vero algoritmo parallelo in quanto i processori devono aspettare le somme parziali.

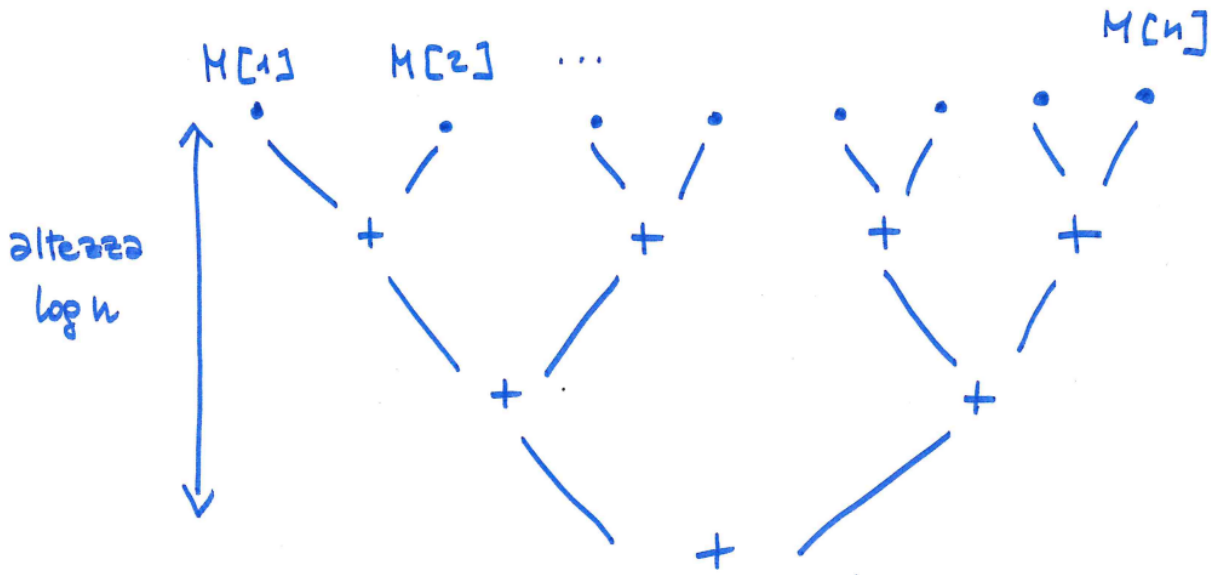
Idea per parallelizzare: "una somma a processore"



Possiamo fare qualcosa di diverso perchè la somma è associativa, potremmo fare:

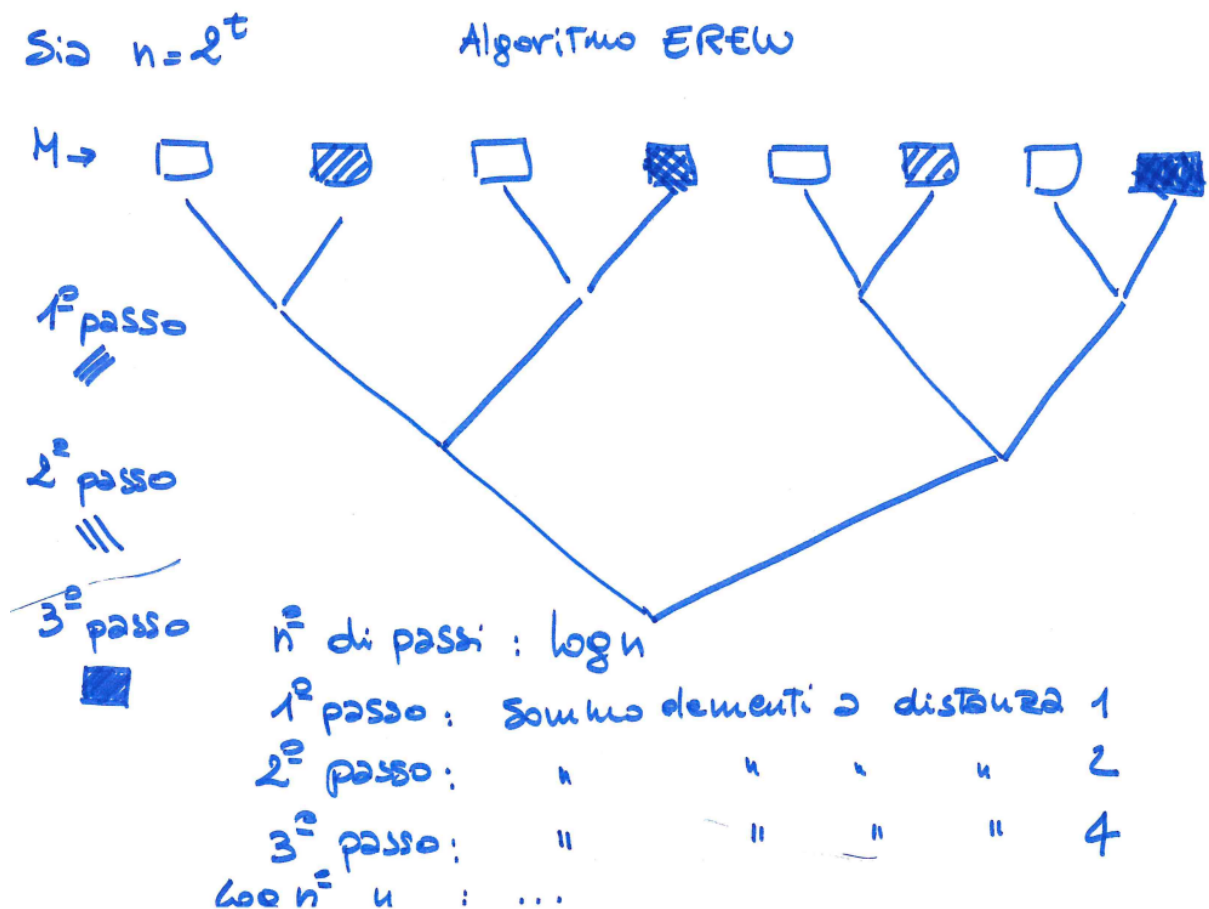
$$((a + b) + c) + d = (a + b) + (c + d)$$

otteniamo così un nuovo algoritmo parallelo, abbiamo un albero le cui foglie sono i dati da sommare, facciamo le somme a due a due, una volta ottenuti questi risultati riappliciamo lo stesso ragionamento perchè le somme sono associative, quindi continuiamo a fare somme in parallelo, poi alla fine calcoliamo il risultato con un unico processore che prenderà in input le due somme delle due metà dei dati di input. Chiaramente stiamo supponendo che n sia potenza di due, vediamo quando n non è potenza di due cosa fare. L'altezza dell'albero che abbiamo costruito è $\log n$.



Supponiamo n potenza di due e vediamo come funziona l'algoritmo EREW, non abbiamo ancora detto come fanno i processori a comunicarsi gli esiti, l'idea è che li vanno a sovrascrivere nelle celle di memoria dell'input. Diciamo che la cella in cui si sovrascrive il risultato parziale è quella con indice più alto. Al secondo passo i processori sanno che i dati su cui fare la somma sono quelli in indice pari, dato questo possiamo notare che

al primo passo sommiamo elementi a distanza 1, al secondo passo sommiamo elementi a distanza due, al terzo passo a distanza 4 e così via, si intuisce che la distanza è una potenza di 2 fino al passo $\log n$.



Scriviamo le istruzioni, consideriamo k l'indice dei processori. Al primo passo si sommano le celle con indice pari con quelli a distanza 1. Al secondo passo nelle celle multiple di quattro devo sommare i numeri a distanza 2:

Istruzioni:

1° passo : $M[2k] = M[2k] + M[2k-1] \quad 1 \leq k \leq \frac{n}{2}$

2° passo : $M[4k] = M[4k] + M[4k-2] \quad 1 \leq k \leq \frac{n}{4}$

3° passo : $M[8k] = M[8k] + M[8k-4] \quad 1 \leq k \leq \frac{n}{8}$

$\vdots$

$\log n$ passo : $M[n] = M[n] + M[n/2] \quad 1$

j è l'indice del processore, 2^{j-1} è la distanza. Il primo for scandisce i passi paralleli, il secondo è un for parallelo che fa lavorare j processori. Il processore k esimo fa una somma

di numeri che stanno ad una certa distanza, se il processore è j la distanza è 2^{j-1} e il risultato lo mette nella cella con indice più alto:

```

1 for i = 1 to log n
2   for k = 1 to n/2^j par do
3     M[2^j k] = M[2^j k] + M[2^j k - 2^{j-1}]
4 return M[n]

```

Ma questo algoritmo è veramente EREW? devo dimostrare che dato che a e b le celle di memoria centrale che utilizza l'algoritmo in parallelo sono diverse:

Processori: a, b processori: $a \neq b$

a $2^j a, 2^j a - 2^{j-1}$
 b $2^j b, 2^j b - 2^{j-1}$

celle coinvolte da a
" da b

confronti:

$2^j a \neq 2^j b$ sì per $a \neq b$
 $2^j a \neq 2^j b - 2^{j-1}$ per assurdo: $\Rightarrow 2a = 2b - 1 \Rightarrow a = \frac{2b-1}{2} \notin \mathbb{N}$

Dobbiamo dimostrare anche la correttezza dell'algoritmo, dobbiamo dimostrare che questa uguaglianza sia corretta:

$$M[2^j k] = M[2^{j-1} k] + \dots + M[2^{j-1} (k-1) + 1]$$

se lo dimostriamo dimostriamo la correttezza per sommatoria, perchè significherebbe che stiamo sommando tutto nella cella M se consideriamo $j = \log n$ e $k = 1$:

$$M[n] = M[n] + \dots + M[1]$$

La dimostrazione va fatta per induzione. Iniziamo dal caso base che è il primo passo parallelo per $j = 1$ e k compreso tra 1 e $\frac{n}{2}$ e l'algoritmo ha questo passo:

```

1 M[2k] = M[k] + M[k-1]

```

Questo significa che la proprietà di prima vale. Adesso dobbiamo fare il passo induttivo, supponiamo che la proprietà sia vera per il passo $j-1$ e dimostriamo che vale anche per j :

PASSO INDUTTIVO: supponiamo vera la (*) per $j-1$
 dimostriamo che vale per j

istruzione $\Rightarrow$ $M[2^j k] = M[2^j k] + M[2^j k - 2^{j-1}]$
 del prog.

$$M[2^{j-1} 2k] = M[2^{j-1} 2k] + \dots + M[2^{j-1} (2k-1) + 1]$$

(*)

$$M[2^{j-1} (2k-1)] = M[2^{j-1} (2k-1)] + \dots + M[2^{j-1} (2k-2) + 1]$$

(*)

$$M[2^j k] = M[2^j k] + \dots + M[2^j (k-1) + 1]$$

$\downarrow$
 la (*) è vera!

2.4.1 Valutazione

Cerchiamo di valutare l'algoritmo. Il livello più costoso in termini di processori è il primo, quindi il numero totale è $\frac{n}{2}$. Il tempo parallelo deve essere logaritmico, andiamo a considerare le micro istruzioni (quella centrale nei for), fa load, load, add e store. Quindi fa 4 istruzioni e il tempo su quattro processori è $4 \log n$.

Un piccolo accorgimento dobbiamo farlo se n non è potenza di 2, possiamo andare a prendere la prima potenza di due più vicina a n aggiungendo zeri. Ma quanto è vicina sta potenza? perchè se devo utilizzare come potenza di 2 2^n non funziona più niente. L'idea per fare vedere che non bisogna aggiungere tanti zeri o un numero esponenziale arriva da questo semplice schemino, proviamo a scrivere n in binario, in binario la potenza di due più vicina avrà un bit in più e tutti i bit a zero, questa potenza di due la troviamo tra n e $2n$, perchè $2n$ in binario è n con uno zero in più in fondo. Quindi tuttalpiù l'input raddoppia.

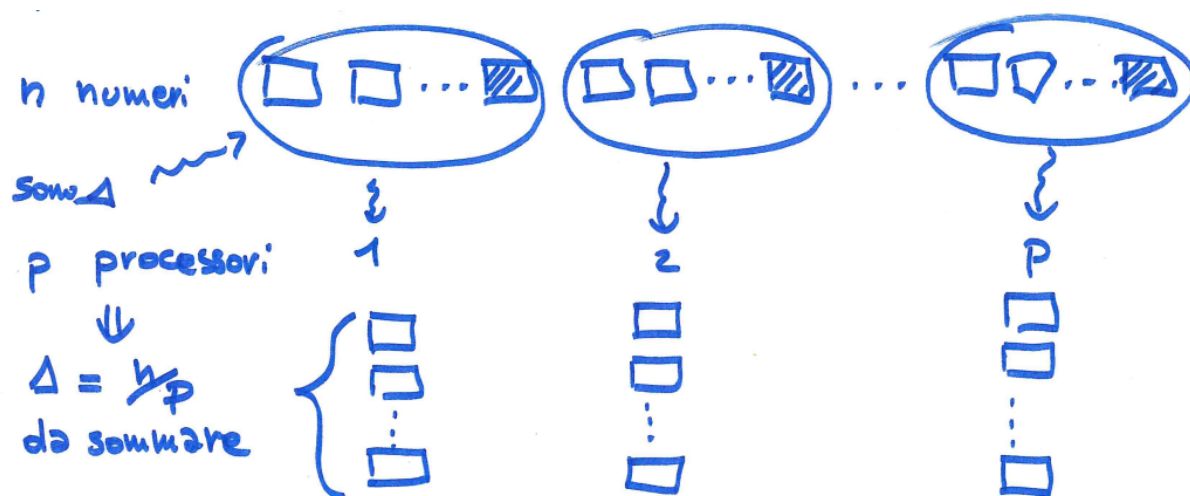
Arriviamo al parametro efficienza, al numeratore abbiamo il tempo sequenziale e sotto il tempo parallelo per il numero di processori:

$$E(n, n) = \frac{n-1}{n 5 \log n} \approx \frac{n}{5 \log n} \rightarrow 0$$

Notiamo che tende a zero lentamente, vogliamo usare meno processori ma lasciare il tempo parallelo logaritmico, tutto questo per avere l'efficienza che tende ad una costante c diversa da zero.

Dobbiamo applicare Wyllie. Invece di avere $n/2$ processori dobbiamo averne p (lasciamolo incognita) e questi processori devono avere in carico più numeri da sommare. Il

numero che hanno in carico lo chiamiamo delta che ha $\frac{n}{p}$ numeri. Quindi ogni processore considera un gruppo di delta elementi ed esegue la somma dei numeri in sequenza. Mettiamo i risultati nella cella coinvolta dalla somma con indice più alto.



1° passo parallelo

$$1 \leq k \leq p \quad M[k\Delta] = M[k\Delta] + \dots + M[(k-1)\Delta + 1]$$

Dopo il primo passo parallelo abbiamo i risultati parziali nelle celle delta fino a $p\Delta$. A queste celle uso l'algoritmo sommatoria visto prima, è vero che nella cella $M[n]$ che è la $M[p\Delta]$ ci finisce tutta la sommatoria.

Proviamo a valutare questo nuovo algoritmo:

- Processori sono p
- il tempo parallelo è composto da un passo sequenziale ed uno parallelo: $T(n, p) = \frac{n}{p} + 5 \log p$
- Efficienza: $E(n, p) = \frac{n-1}{n+5p \log p}$. Ci piacerebbe avere $5p \log p = n$ in modo da avere un'efficienza costante che sarebbe $1/2$.

Siamo riusciti in pieno a utilizzare il principio di Wyllie.

Il nostro algoritmo per sommatoria è logaritmico, ma possiamo fare di meglio? possiamo dimostrare un lower bound. Utilizzando l'albero binario per visualizzare l'algoritmo parallelo dimostriamo che un lower bound per la sommatoria è il logaritmico.

$$\text{Vorrei } P \log p = \frac{n}{5} \leadsto P = \frac{n}{\log n \cdot 5}$$

$$\begin{aligned} \text{Infatti: } \frac{n}{5 \log n} \left(\log \frac{n}{(\log n)^5} \right) &= \frac{n}{5 \log n} (\log n - \log 5 - \log \log n) \\ &= \frac{n}{5} \left(1 - \frac{\log 5}{\log n} - \frac{\log \log n}{\log n} \right) \sim \frac{n}{5} \end{aligned}$$

Ricapitolando:

$$P(n) = \frac{n}{5 \log n}$$

$$T(n, P(n)) = 5 \log n + 5 \log n - \dots \leq 10 \log n$$

[Uso un numero di processori sublineare e mantengo un tempo logaritmico] Wyllie

2.5 OP Iterata

La sommatoria è un caso particolare di questo problema. Ho in input n valori e voglio calcolare l'operazione OP su questi valori e voglio mettere il risultato nella cella di indice maggiore. Quando OP è la somma abbiamo proprio il problema sommatoria, ricordiamo però che OP deve godere della proprietà associativa. Quindi possiamo scegliere operazioni di somma, moltiplicazione, operazioni booleane, minimo, massimo e la concatenazione.

2.5.1 CRCW - PRAM

Supponiamo di avere una PRAM CRCW e di focalizzarci sull'AND iterato, qui viene fornito un programma che risolve il problema in tempo costante:

```

1 for i <= k <= n par do
2   if M[k] = 0 term
3     M[n] = 0

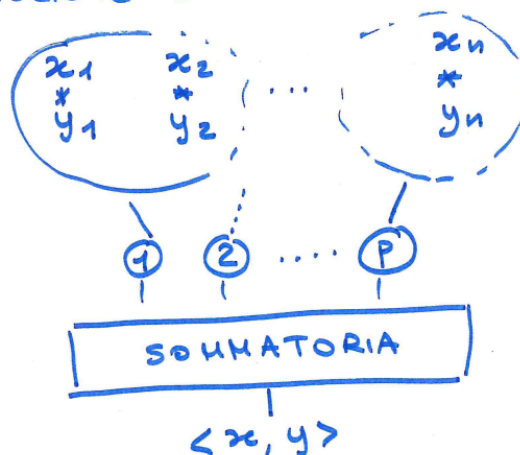
```

Nell'and binario se c'è uno zero sa che il risultato totale è 0. L'if che fa un test e la scrittura esegue delle write simultanee nella cella di indice n . Il numero di processori è n . Il tempo parallelo è costante $T(n, n) = 3$, quando calcolo l'efficienza ottengo $E(n, n) = \frac{n-1}{n \cdot 3} \rightarrow \frac{1}{3}$.

2.6 Prodotto interno di vettori

Consiste nel prendere le componenti di x e y e farne il prodotto in questo modo: $\langle x, y \rangle = \sum_{i=1}^n x_i \cdot y_i$ e poi fatta la somma componente per componente sommiamo tutto. Ha come sottoproblema la sommatoria. Facciamo prima gli n prodotti e poi la sommatoria. Il tempo sequenziale è $2n - 1$.

Soluzione EREW:



Vediamo come poter utilizzare l'algoritmo della sommatoria in questo problema del prodotto interno. Raggruppiamo i numeri ottenuti dai prodotti in gruppi di cardinalità $\log n$ e li affidiamo ad un processore. Qui il processore deve fare anche il prodotto. Il primo passo richiede di fare i prodotti in sequenza (i prodotti sono $\log n$), dopo di che deve fare la somma sequenziale di quello che ha ottenuto. Questo è un algoritmo EREW efficiente.

Per sommatoria:

$$P = c_1 \frac{n}{\log n} \quad t_{II} = c_2 \log n \quad \left. \vphantom{\begin{matrix} P \\ t_{II} \end{matrix}} \right\} \text{II fase}$$

$$P \approx \frac{n}{\log n} \Rightarrow \Delta = \frac{n}{P} = \log n \Rightarrow t_I = c_3 \log n \quad \left. \vphantom{\begin{matrix} P \\ \Delta \end{matrix}} \right\} \text{I fase}$$

Riassumendo:

$$\begin{aligned} \langle x, y \rangle \text{ costo: } P &\approx \frac{n}{\log n} & t &= t_I + t_{II} \approx \log n \\ E &\sim \frac{2n-1}{\frac{n}{\log n} \cdot \log n} \rightarrow C \neq 0 \text{ costante} \end{aligned}$$

2.6.1 Prodotto matrice vettore

Abbiamo in input una matrice A di dimensione n e un vettore x . Vogliamo calcolare Ax . Dobbiamo moltiplicare ogni riga per ogni colonna. Il tempo sequenziale è $2n^2 - n$. Posso pensare di fare in parallelo i prodotti interni in parallelo, uso il modulo prodotto

interno in parallelo n -volte. Attenzione che così non abbiamo più un algoritmo EREW perchè il vettore x è acceduto simultaneamente dai moduli, quindi l'algoritmo che stiamo progettando è un CREW.

$$\begin{aligned}
 P(n) &\approx n \cdot \frac{n}{\log n} & T(n, P(n)) &\approx \log n \\
 E(n, P(n)) &\sim \frac{n^2}{\frac{n^2}{\log n} \cdot \log n} \rightarrow C \neq 0 \text{ costante}
 \end{aligned}$$

2.6.2 Prodotto MATRICE MATRICE

In input abbiamo due matrici quadrate A, B . In output desideriamo il prodotto AB , vogliamo calcolare n^2 prodotti vettoriali in parallelo. Attenzione che ogni riga di A e colonna di B viene acceduta simultaneamente, abbiamo quindi un algoritmo CREW.

$$\begin{aligned}
 \text{Prestazioni:} \\
 P(n) &\approx n^2 \cdot \frac{n}{\log n} & T(n, P(n)) &\approx \log n \\
 E(n, P(n)) &\sim \frac{n^{2,8}}{\frac{n^3}{\log n} \cdot \log n} = \frac{n^{2,8}}{n^3} \rightarrow 0 \text{ lentamente}
 \end{aligned}$$

2.6.3 Potenza di matrice

In input abbiamo una matrice e in output la potenza della matrice che è potenza di due.

Esiste un algoritmo sequenziale che calcola velocemente la potenza di una matrice se questa è potenza di due:

```

1 for i = 1 to log n do
2   A = A x A

```

calcola le potenze solo di matrici in potenza di due.

Per farlo sequenziale sfrutto questa idea, solo che invece di usare un prodotto sequenziale lo faccio parallelo, come visto prima.

$$\begin{aligned}
 \text{prestazioni:} \\
 P(n) &= \frac{n^3}{\log n} & T(n, P(n)) &= \log n \cdot \log n = \log^2 n \\
 E &\sim \frac{n^{2,8} \cdot \log n}{\frac{n^3}{\log n} \cdot \log^2 n} = \frac{n^{2,8}}{n^3} \rightarrow 0 \text{ lentamente}
 \end{aligned}$$

2.6.4 Somme Prefisse

Anche lui contiene al suo interno il problema della sommatoria. Ci sono varie applicazioni che usano questo problema come analizzatori lessicali nei compilatori, problemi di ricerca

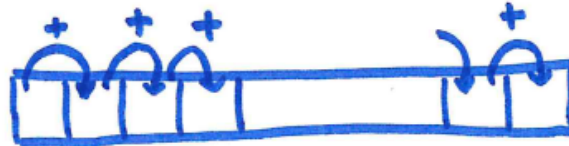
di parole nei testi. Si presenta come input n valori (numeri) e come output la somma di tutti i numeri precedenti : $\sum_{i=1}^k M[i] \rightarrow M[k]$. Il problema richiede di calcolare il prefisso di ogni cella, con prefisso intendiamo la somma dei numeri precedenti. Esiste un algoritmo sequenziale per risolvere questo problema:

```

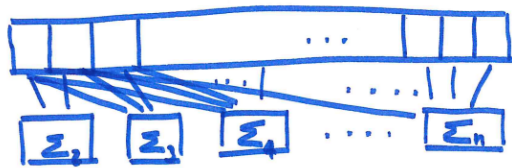
1 for k = 2 to n do
2   M[k] = M[k] + M[k-1]

```

Si prende il valore della cella, lo si somma al successivo e si mette la somma nella cella. Si va avanti così, il tempo di questo algoritmo sequenziale è $n-1$.



Una prima proposta per farlo parallelo potrebbe essere quello di fare la sommatoria di tutti i possibili prefissi. Prende in input tutti gli elementi e applica il modulo sommatoria. Il risultato lo metterà nella cella di indice più alto.



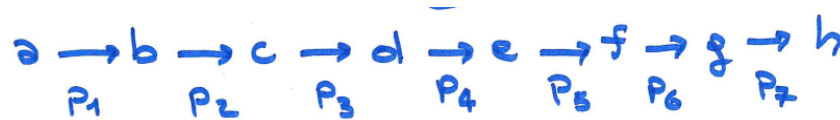
Risolve con
SOMMATORIA
tutti i possibili
prefissi.

Dopo aver calcolato l'efficienza notiamo che la proposta fatta non ci soddisfa.

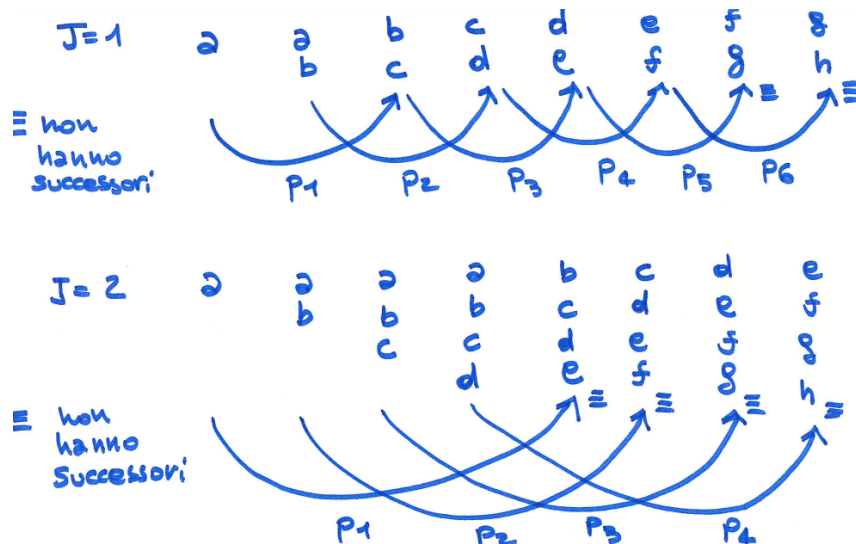
$$\begin{aligned}
 p(n) &\leq (n-1) \cdot \frac{n}{\log n} \sim \frac{n^2}{\log n} \\
 &= \sum_{i=2}^n i / \log i \geq \frac{1}{\log n} \sum_{i=2}^n i \sim \frac{n^2}{\log n} \\
 T(n, p(n)) &\sim \log n \quad E \sim \frac{n-1}{\frac{n^2}{\log n} \cdot \log n} \rightarrow 0 \\
 &\quad \text{POCO EFFICIENTE}
 \end{aligned}$$

Pointer Doubling

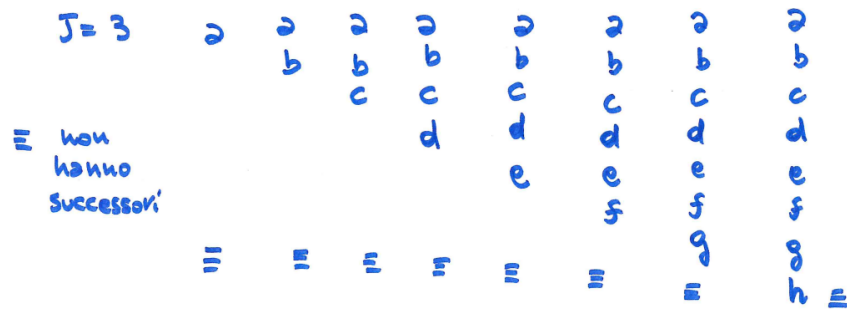
Per avere qualcosa di meglio non dobbiamo usare banalmente il problema sommatoria, ma dobbiamo inventarci qualcosa di nuovo. L'ha inventato Kogge-Stone e la tecnica si chiama Pointer doubling. Sono dei puntatori tra coppie di numeri, noi li indichiamo con una freccia. Se mettiamo un link tra i due numeri ci deve essere un processore che li somma e mette la somma da qualche parte, mette la somma nella cella con indice più alto.



Supponiamo di avere 8 numeri che vengono indicati con delle lettere, inizialmente poniamo dei link tra una cella e il suo successivo, come detto in precedenza un processore si occupa di una freccia, prende l'elemento da cui parte la freccia, lo somma con l'elemento a cui arriva la freccia e mette la somma nell'indirizzo più alto.



Sempre al passo $J = 1$ vengono aggiornati i link, vengono raddoppiati i link, invece di andare da un elemento al suo successivo viene messo un link a distanza 2. Diremo anche che la somma (bc) è il successore di a , il link mette il legame tra due numeri e a questo punto si può dire che uno diventa il successore dell'altro. Dobbiamo costatare che per gli ultimi due elementi non abbiamo successori, quindi non è possibile mettere un link che parte da loro. Aggiorniamo ancora le celle con le nuove somme. Al passo $J = 2$, dato che alcune somme prefisse non sono complete cambiamo ancora i link raddoppiando la distanza di prima, quindi mettiamo il link a distanza 4.



Al passo $J = 3$ abbiamo finito, ci accorgiamo che abbiamo finito perchè per tutti i successori non è possibile mettere archi (nel nostro caso dovremmo mettere link a distanza 8). E notiamo che abbiamo risolto correttamente il problema delle somme prefisse.

Per scrivere il codice proviamo a rispondere a queste domande:

- Quanti numeri privi di successore genera il passo j -esimo? 2^j
- Quanti passi dura l'algoritmo? fintanto che ho successori: $\log n = 2^j = n \rightarrow j = \log n$
- quali processori attivo al j -esimo passo? $1 \leq k \leq n - 2^{j-1}$
- Sia $S[k]$ la posizione del successivo di $M[k]$ come inizializzo S ? $S[k] = k + 1$ e $S[n] = 0$. Condividiamo in memoria anche il vettore dei successori oltre alle celle M con l'input.
- Dato P_k , quale istruzione su M deve eseguire? $M[k] + M[S[k]] \rightarrow M[S[k]]$. Successivamente devo fare l'aggiornamento del vettore S : $S[k] = (S[k] == 0 ? o : S[S[k]])$. Quello che facciamo è invece che raddoppiare le distanze facciamo puntare il successore al successore di quel numero.

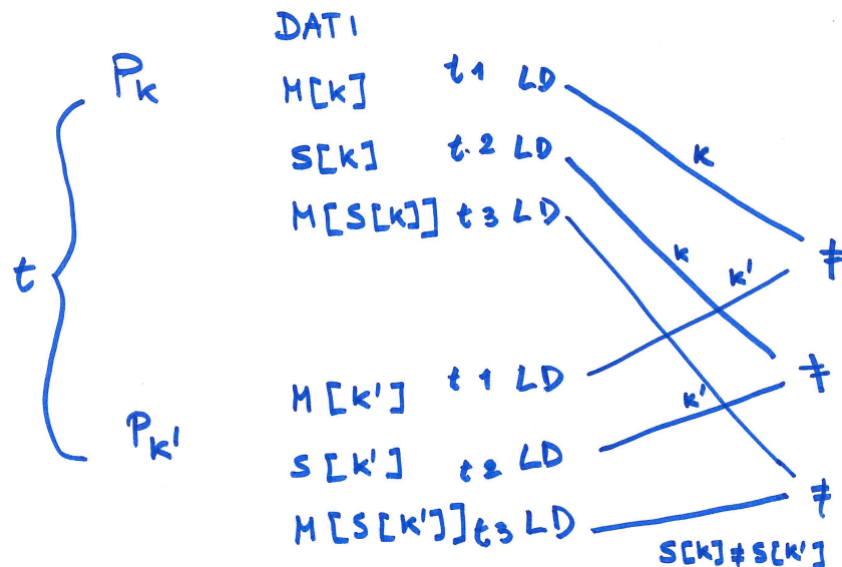
Vediamo ora il codice, il numero di passi sono $\log n$:

```

1 for j = 1 To logn do
2   for 1 <= k <= n - 2^{j - 1} par do
3     M[S[k]] = M[k] + M[S[k]]
4     S[k] = (S[k] == 0 ? o : S[S[k]])

```

Guardando questi accessi alle celle possiamo chiederci se i processori competono all'accesso della stessa cella? NO, stiamo risolvendo un problema EREW.



Può essere che due processori considerino la stessa cella con idee diverse, ma quando succede succede sempre a tempi diversi.

Vediamo la valutazione. I processori sono $n-1$. Il tempo parallelo, ci sono due for, quello interno parallelo fa eseguire due istruzioni. Scomponiamo queste istruzioni in microistruzioni e troviamo che in totale abbiamo 9 istruzioni che vengono eseguite per $\log n$:

$$\begin{aligned}
 P(n) &= n-1 \\
 T(n, P(n)) &= \begin{aligned} &- M[S[k]] = M[k] + M[S[k]] \\ &5 \left\{ \begin{array}{l} \text{LOAD } M[k] \\ \text{LOAD } S[k] \\ \text{LOAD } M[S[k]] \\ \text{ADD} \\ \text{STORE } M[S[k]] \end{array} \right. \\ &- S[k] = (S[k] == 0 ? 0 : S[S[k]]) \end{aligned} \\
 &\quad \begin{array}{l} J=1, \dots, \log n \\ \downarrow \\ T(n, n-1) \sim 9 \log n \end{array} \quad 4 \left\{ \begin{array}{l} \text{LOAD } S[k] \\ \text{JZERO} \\ \text{LOAD } S[S[k]] \\ \text{STORE } S[k] \end{array} \right. \\
 E(n, P(n)) &= \frac{n-1}{(n-1) 9 \log n} \Rightarrow \frac{1}{9 \log n} \rightarrow 0 \text{ lentamente}
 \end{aligned}$$

Valutiamo l'efficienza:

$$E(n, p(n)) = \frac{n-1}{(n-1)9 \log n} = \frac{1}{9 \log n} \rightarrow 0$$

e scopriamo che tende a 0 lentamente, quindi vediamo se riusciamo a migliorare questo algoritmo, forse stiamo usando troppi processori.

Quindi sfruttiamo Wyllie come per la sommatoria (per far sparire la funzione $\log n$ da E), quindi raggruppiamo i numeri in gruppi e applichiamo un algoritmo sequenziale:

$$p(n) = o\left(\frac{n}{\log n} T(n, p(n))\right) = O(\log n)E \rightarrow C \neq 0$$

Come per sommatoria anche l'algoritmo date per le SOMME PREFISSE può essere usato per OP-PREFISSA in cui in input abbiamo una sequenza di numeri e in output vogliamo una sequenza di OP dell'input. L'OP può esserci il prodotto, l'and, or, il minimo e massimo e la concatenazione. Basta che l'operazione sia associativa.

2.6.5 Valutazione dei polinomi

Sfrutteremo l'OP prefissa. In input c'è un polinomio di grado n ($p(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$) e un valore α , vogliamo valutare il polinomio $p(x)$ sul valore α . Quindi si sostituisce a x α e si eseguono tutte le operazioni. In Output vogliamo $p(\alpha)$.

In memoria abbiamo il valore α che sta in una cella, e poi abbiamo il polinomio che è specificato con i coefficienti a_n nella memoria condivisa, le celle in cui mettiamo i coefficienti le chiamiamo A , $A[0], \dots, A[n]$.

Se voglio valutare il numero di operazioni per ottenere $p(\alpha)$ ottengo che il numero di operazioni tra prodotti e somme è nell'ordine di n^2 .

In realtà si può fare di meglio grazie all'idea di Ruffini-Horner (raccoliere x in maniera iterata):

Miglioramento di Ruffini - Horner

idea : $p(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + a_4x^4$

↓
raccolti
x in maniera
iterata

$$= a_0 + x(a_1 + a_2x + a_3x^2 + a_4x^3)$$

$$= a_0 + x(a_1 + x(a_2 + a_3x + a_4x^2))$$

$$= a_0 + x(a_1 + x(a_2 + x(a_3 + a_4x)))$$

generalizziamo :

$$p(x) = a_0 + x(a_1 + \dots + x(a_{n-2} + x(a_{n-1} + a_nx)))$$

Diagramma di calcolo iterativo:

```

      a_{n-2} + x(a_{n-1} + a_nx)
      ↑           ↑
      + α       + α
      └───┬───┘
          P   P
          └───┬───┘
              P
  
```

$p = a_j + p \cdot \alpha$

Questa scrittura suggerisce una soluzione, ad x sostituisce α , inizializza una variabile p ad a_n .

Vediamo l'algoritmo:

```

1 Input (alpha)
2 p = an
3 for i = 1 to n
4   p = a{n-1} + p per alpha
5 Output(p)
  
```


Le operazioni sono due per il numero di iterazioni per il for: $T(n, 1) = 2n$.
Vediamo un possibile algoritmo parallelo:

- Costruisci il vettore delle potenze di α : $Q[k] = \alpha^k$.
- Eseguo il prodotto interno $\langle A, Q \rangle$. $\langle A, Q \rangle = \sum_{k=0}^n A[k] \cdot Q[k]$
- Restituisco $\langle A, Q \rangle$

Per risolvere il punto 1:

- metto α in tutti gli elementi di Q da 1 a n : $Q[1] = \alpha, \dots, Q[n] = \alpha$. Questo richiede di risolvere il problema REPLICA
- Applico il PRODOTTO-PREFISSO su Q (cosa che abbiamo già visto funzionare bene in parallelo): $Q[1] = \alpha, Q[2] = \alpha^2, \dots, Q[n] = \alpha^n$

Adesso dobbiamo risolvere il problema replica in parallelo. Il primo modo è utilizzare una batteria di n processori ognuno si occupa di sistemare una cella:

```

1 for k = 1 to n par do
2   Q[k] = alpha

```

È un algoritmo parallelo che può essere eseguito su una CREW, perchè abbiamo una lettura concorrente su α . Prestazioni:

- $p = n$
- $t = 2$
- $E \approx \frac{n}{n \cdot 2} \rightarrow c \neq 0$

Per abbassare il numero di processori posso usare Wyllie raggruppando i numeri in gruppi di $\log n$, ogni processore si occupa di replicare α in tutte quelle celle. Il k -esimo processore carica α nelle celle di posizione $(k-1)\log n + 1, \dots, k\log n$. Vediamo il codice:

```

1 for k = 1 to n/log n par do
2   for i = 1 to log n do
3     Q[(k-1) log n + i] = alpha

```

Prestazioni:

- $p = \frac{n}{\log n}$
- $t = c \log n$
- $E = \frac{n}{\frac{n}{\log n} \cdot c \log n} = \frac{1}{c} \neq 0$. Risulta essere efficiente.

Abbiamo risolto l'utilizzo eccessivo di processori ma abbiamo comunque un algoritmo CREW.

Un modo per poter ottenere un replica EREW è inizializzare un vettore con primo elemento α e tutti zeri, e poi fare in modo che sia replicato. Come è possibile farlo? basta eseguire il modulo somme prefisse:

```

1 Input(alpha)
2 Q[1] = alpha
3 for k = 2 to n par do
4   Q[k] = 0

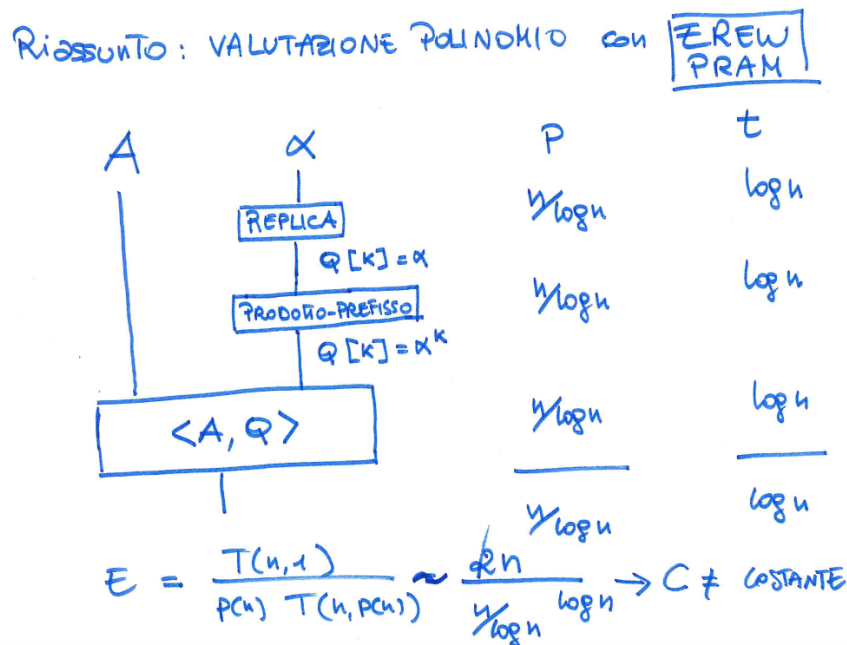
```

La differenza qui è che 0 è una costante, i processori non hanno bisogno di accedere in memoria per replicarla.

Le prestazioni di questi passi sono:

1. $p = \frac{n}{\log n}$ $t = \log n$. Usando Wyllie
2. $p = \frac{n}{\log n}$ $t = \log n$

Quindi riesco ad ottenere un algoritmo efficiente, esattamente come prima, con la differenza che qui abbiamo un algoritmo EREW.



2.7 Ricerca di un elemento

Ho un array di n elementi ($M[0], \dots, M[n]$) e in input un valore α , voglio sapere se il valore è presente nell'array. Se scopro che c'è vado a posizionare nell'ultima cella del vettore il valore 1, 0 altrimenti. L'algoritmo sequenziale classico richiede n passi, chiaramente se l'input è ordinato posso migliorare il numero di passi arrivando a $\log n$ attraverso la ricerca dicotomica, confrontiamo il valore con quello centrale, se è maggiore ci spostiamo nella seconda metà e confrontiamo il valore centrale di questa metà (a sx se minore) e l'operazione si ripete.

È stato presentato un algoritmo quantistico che dimostra che su input non ordinato dobbiamo usare $\sqrt{n}$ passi. Al momento non abbiamo computer quantistici generici.

Vediamo un primo algoritmo parallelo su architettura CRCW:

```

1 F = 0
2 for k = 1 to n par do
3     if (M[k] == alpha)
4         F = 1
5 M[n] = F

```

Ogni processore controlla una cella di M, se è uguale al valore di alpha mette il flag a 1. Prestazioni:

- $p = n$
- $t = costante$

Trasformiamo il vettore M di ricerca in un vettore booleano, quando una cella contiene α faccio scrivere 1, 0 altrimenti: $M[k] = (M[k] == \alpha ? 1 : 0)$. Poi alla fine faccio un MAX-ITERATO per spostare il flag 1 nell'ultima cella. Prestazioni:

- $p = n$
- $t = costante$ ma se applico Wyllie: $p = \frac{n}{\log n} t = \log n$

Questo mi dà un'efficienza costante diversa da zero. Adesso abbiamo ottenuto un CREW, la lettura concorrente è data dall'accesso simultaneo alla cella α .

Si può risolvere mettendo un vettore con tutte repliche di α .

EREW:

1. REPLICA $\alpha \rightarrow A[1], \dots, A[n]$
2. for k = 1 to n par do { $M[k] = (M[k] == A[k] ? 1 : 0)$ }
3. MAX-ITERATO ($M[1], \dots, M[n]$)

Prestazioni:

1. $p = \frac{n}{\log n} t = \log n$
2. $p = \frac{n}{\log n} t = \log n$
3. $p = \frac{n}{\log n} t = \log n$

Tot: $p = \frac{n}{\log n} t = \log n$ $E = Costante \neq 0$.

2.7.1 Varianti per problemi legati

Possiamo cambiare leggermente la domanda che ci poniamo e risolvere altri problemi con lo stesso codice:

- Quante volte α è contenuto nel vettore M: il punto 3 diventa SOMMATORIA
- Posizione massima di α in M: il punto 2 diventa $M[k] = k$ se c'è α . Scrivo l'indice della cella se è contenuto.
- Posizione minima di α in M: come sopra modifico il punto 2 e il punto 3 diventa OP-ITERATA

2.8 Ordinamento

Da prima vedremo un algoritmo di ordinamento non molto efficiente che si basa sul counting sort. Abbiamo in input n valori e desideriamo in output la sequenza ordinata, potremmo chiedere che sia ordinate nelle stesse celle oppure una permutazione dell'input in cui l'indice ci dice in che posizione va l'elemento nella sequenza.

Gli algoritmi che permettono di fare ordinamenti sono basati su confronti, un confronto è una domanda del tipo $M[j] \leq M[i]$? Hanno un tempo sequenziale $n \log n$ che è sia un upper bound ed un lower bound (meno di così non possiamo fare per algoritmi basati su confronti). L'idea del perchè sia un lower bound è basato sull'albero di decisione, qualunque algoritmo di ordinamento basato su confronti genera un cammino di un albero di decisione. Sui nodi abbiamo i confronti. Le foglie rappresentano la permutazione dell'input p . Una foglia individua un unico cammino a partire dalla radice e quindi i confronti che mi permettono di ordinare l'input. L'altezza dell'albero rappresenta il tempo dell'algoritmo di ordinamento, perchè è il cammino più lungo.

Osserviamo che il numero di foglie deve essere $n!$ (prevedere tutte le permutazioni di n elementi), se t è l'altezza dell'albero di decisione so che vale la seguente relazione: il numero di foglie non deve superare 2^t dove t è l'altezza, da cui si ricava che t deve essere almeno $\log n!$. Da qui ci sono una serie di conti che portano a vedere che l'altezza è circa $n \log n$.

Vediamo un primo approccio parallelo che si basa sul conteggi di alcuni confronti, sequenzialmente richiederebbe $\Theta(n^2)$. Noi prendiamo spunto dal counting sort. Assumiamo che n sia una potenza di 2 e che gli elementi siano tutti diversi tra loro.

Counting sort conta i confronti che coinvolgono l'array: l'elemento $M[i]$ va in k solo se ho k elementi minori uguali di $M[i]$.

Useremo un vettore V per posizionare i valori ordinati del vettore M .

Vediamo l'algoritmo sequenziale come è fatto, vuole contare quanti elementi minori uguali di esso ci sono, in questo modo riempie la cella di v (v è il set di contatori che permettono di memorizzare gli elementi minori uguali). Con il primo for inizializza a zero tutti i contatori. Per ogni elementi $M[i]$ controlla tutti gli elementi della sequenza e se è minore uguale incremento il vettore in posizione i , mi dice quanti sono minori uguali di i . Se la soluzione che vogliamo dare è il vettore V l'algoritmo finisce qui, altrimenti dobbiamo mettere $M[i]$ nella posizione $V[i]$ allora utilizzo un array F che mi dà l'array M ordinato è di appoggio perchè poi carico le celle di F in M :

```
1 for i = 1 to n
2   V[i] = 0
3
4 for i = 1 to n
5   for j = 1 to n
6     if (M[j] <= M[i]) then V[i]++
7
8 for i = 1 to n
9   F[V[i]] = M[i]
10
11 for i = 1 to n
12   M[i] = F[i]
```

Il tempo è n^2 .

Per parallelizzarlo possiamo pesare di prendere un numero di processori n^2 ed utilizzare un processore per fare un confronto e mette la risposta in una matrice booleana posizionando 1 se è minore 0 altrimenti: $V[i, j] = (M[j] \leq M[i] ? 1 : 0)$. La riga i -esima rappresenta gli elementi della matrice che sono minori uguali a $M[i]$.

Se per ogni i eseguo la sommatoria parallela della i -esima riga ottengo nell'ultima colonna della matrice la posizione che gli elementi devono assumere.

Vediamo il codice:

```

1 for i <= i, j <= n par do
2   V[i, j] = (M[j] <= M[i] ? 1 : 0)
3
4 for i = 1 to n par do
5   SOMMATORIA(V[i, 1], ..., V[i, n])
6
7 for i = 1 to n par do
8   M[V[i, n]] = M[i]

```

Abbiamo ottenuto un CREW, ma replicando i dati possiamo arrivare ad un EREW.

Prestazioni : proc. / tempo / effic. CREW $\rightarrow$ EREW

Fase 1) $P(n) = n^2$ $T(n, n^2) = 4$ (LD, LD, J², ST)
 al solito posso ottenere:
 Fase 2) $P(n) = \frac{n^2}{\log n}$ $T \sim \log n$
 n moduli SOMMATORIA $P = \frac{n}{\log n}$ $t = \log n$
 $P(n) = \frac{n^2}{\log n}$ $T \sim \log n$
 Fase 3) $P(n) = n$ $T = 3$ (LD, LD, ST)

TOT. $P \sim \frac{n^2}{\log n}$ $T \sim \log n$

$$E = \frac{n \log n}{\frac{n^2}{\log n} \cdot \log n} = \frac{\log n}{n} \rightarrow 0$$

Abbiamo visto un algoritmo che ci ha portato ad un efficienza non molto buona, dobbiamo quindi introdurre delle nuove idee per risolvere dei problemi in parallelo. Una di queste è il bit sort, mette in campo idea di sequenze bitonali.

2.8.1 Bit-Sort

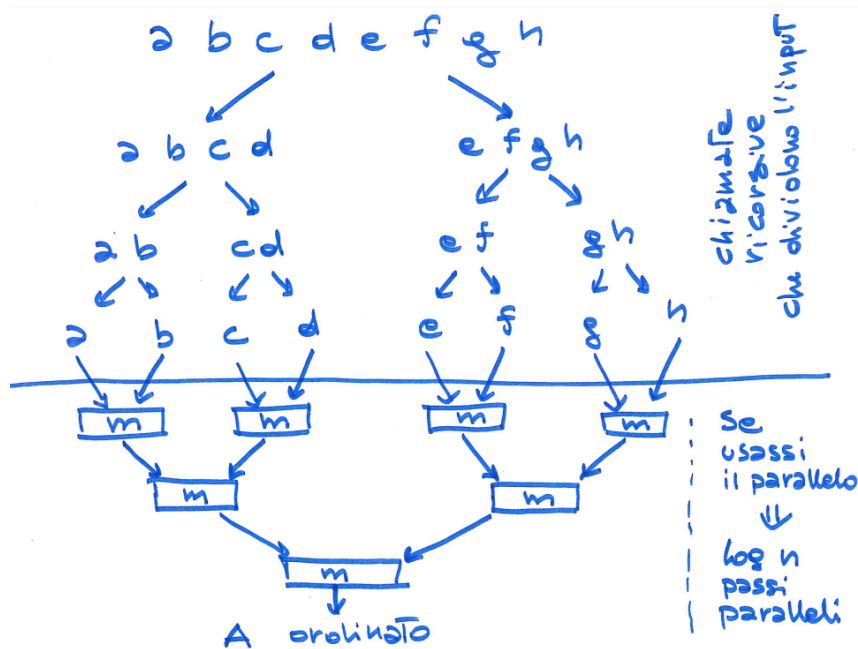
Prende spunto dal merge sort il quale usa la tecnica divide et impera, la procedura è di prendere una sequenza, controllare se è maggiore di uno, divido in array di n mezzi elementi, passo questa sequenza di nuovo al mergesort. Poi faccio la funzione merge che fonde due array ordinati.

```

1 Mergesort(A[1], ..., A[n]) {
2   if (|A| > 1) {
3     As = Mergesort(A[1], ..., A[n/2])
4     Ad = Mergesort(A[n/2 + 1], ..., A[n])
5     A = Merge(As, Ad)
6   }
7   return A
8 }

```

Il merge confronta i numeri sequenzialmente ed estrae il numero più piccolo. Costa nel caso peggiore n .



Se usassi il parallelo mi costerebbe solo la funzione merge, sarebbero $\log n$ passi paralleli che utilizzano il merge. Il problema è che merge non è parallelizzabile ed il costo massimo è n . Otterrei quindi un tempo parallelo $n \log n$ (uguale a quello sequenziale).

Basta cambiare la funzione merge, passo alla funzione merge due array completamente ordinati e voglio un array lungo il doppio completamente ordinato. Salta fuori il bit sort, ci chiediamo quando merge risulta essere molto facile e richiede poche operazioni? quando gli elementi del primo sub-array sono minori del secondo perchè in questo caso la fusione sarebbe la concatenazione degli array. Dobbiamo trovare il modo di trovarci in questa situazione. Dobbiamo fare uso di sequenze numeriche BITONICHE. Abbiamo bisogno anche di ulteriori routine parallelo:

- REV: reverse, si scambiano i valori seguendo le frecce, sempre in parallelo. Questa funzione scive il contrario della sequenza stessa
- MINMAX: considera l'array in due metà e confronta gli elementi a coppie, prendendone due a distanza $n/2$ (due di metà diverse). Da questo confronto viene fuori l'elemento minimo (messo in $A[k]$) e massimo (messo in $A[k + n/2]$). Chiamiamo la prima metà A_{min} e la seconda metà A_{max} . Questa funzione ritorna A_{min} concatenato con A_{max} .

- $REV(A[1], A[2], \dots, A[n])$

$A[1] \leftarrow A[n] \quad A[2] \leftarrow A[n-1], \dots, A[n] \leftarrow A[1]$

- $MINMAX(A[1], A[2], \dots, A[n])$

$A[1], \dots, A[k], \dots, A[\frac{n}{2}], \dots, A[k+\frac{n}{2}], \dots, A[n]$
 $\uparrow \qquad \qquad \qquad \uparrow$
 $\min\{A[k], A[k+\frac{n}{2}]\} \quad \max\{A[k], A[k+\frac{n}{2}]\}$
 $\underbrace{\hspace{10em}}_{A_{min}} \quad \underbrace{\hspace{10em}}_{A_{max}}$

Vediamo le procedure di queste operazioni in parallelo:

```

1 Rev(A) {
2   for i <= k <= n/2 par do {
3     SWAP(A[k], A[n - k + 1])
4   }
5 }
6
7 MINMAX(A) {
8   for 1 <= k <= n/2 par do {
9     if (A[k] > A[k + n/2])
10      SWAP(A[k], A[k + n/2])
11   }
12 }
```

In entrambi i casi i processori sono $p = \frac{n}{2}$ ed in entrambi i casi abbiamo tempo costante.

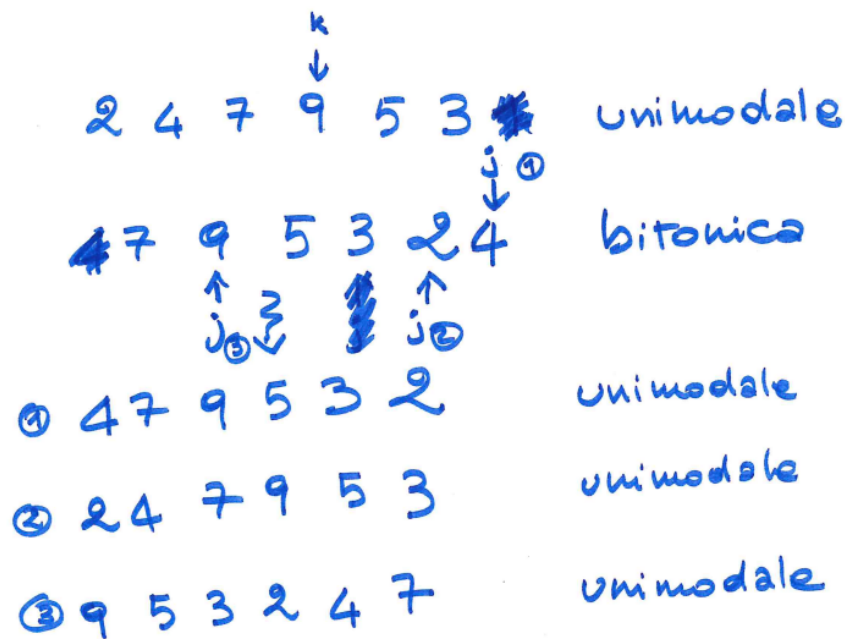
Dobbiamo definire due particolari sequenze numeriche, unimodali e bitoniche. Queste permettono di far funzionare bene il bit-sort. Una sequenza è detta unimodale se e solo se esiste un k che fa da elemento minimo della sequenza, da 1 a k gli elementi decrescono e da k a n gli elementi crescono, oppure è vero il viceversa, k è il massimo quindi prima crescono fino a k e poi decrescono fino a n :

$$A[1] > A[2] > \dots > A[k] < A[k+1] < \dots < A[n]$$

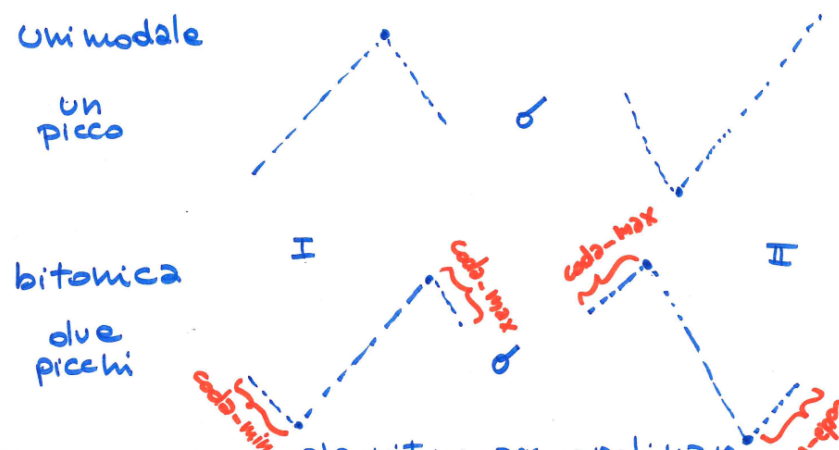
Una sequenza è detta bitonica quando può essere trasformata in unimodale tramite una permutazione ciclica di se stessa, detto formalmente è bitonica se e solo se esiste una permutazione ciclica di se stessa che mi da una sequenza unimodale, ovvero esiste un j tale per cui prendendo gli elementi da j a n concatenati da 1 a $j-1$ ho una sequenza unimodale (significa prendere un elemento e spostarlo per creare una sequenza unimodale, da suffisso a prefisso):

$$A[j], \dots, A[n], A[1], \dots, A[j-1]$$

Vediamo una rappresentazione grafica delle sequenze unimodale e bitoniche. La sequenza si dice unimodale se abbiamo un picco, gli elementi crescono arrivando al massimo



e poi decrescono. È bitonica quando i picchi sono due e gli elementi finali e iniziali devono soddisfare un criterio, se i picchi sono minimo e massimo, gli elementi che sono nella fase decrescente devono essere maggiori di quelli decrescenti all'inizio. Una situazione analoga accade per le bitoniche che hanno prima un massimo e poi un minimo. Se ho queste caratteristiche riesco ad ottenere da una sequenza bitonica una unimodale.



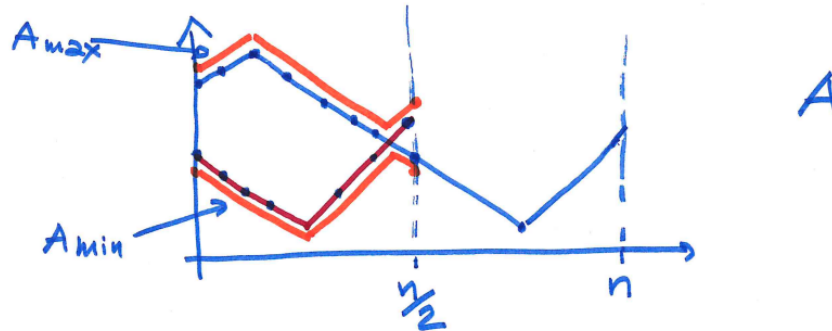
Diamo un algoritmo BIT-MERGE che è un algoritmo per l'ordinamento di sequenze unimodali. Vediamo delle osservazioni:

- Una sequenza unimodale è anche bitonica grazie alla permutazione identità
- Gli elementi di fine array devono essere maggiori degli elementi di inizio array per la forma I e viceversa per la forma II
- Se prendo due sequenze ordinate A, B (crescenti o decrescenti) e concateno A al REV(B) ottengo una sequenza unimodale.

Soa A bitonica, eseguo $\text{MINMAX}(A)$ ottengo:

- $A_{\min}$ e $A_{\max}$ sono bitoniche
- Ogni elemento di $A_{\min}$ è minore di ogni elemento di $A_{\max}$

La dimostrazione di questa proprietà la abbiamo graficamente:



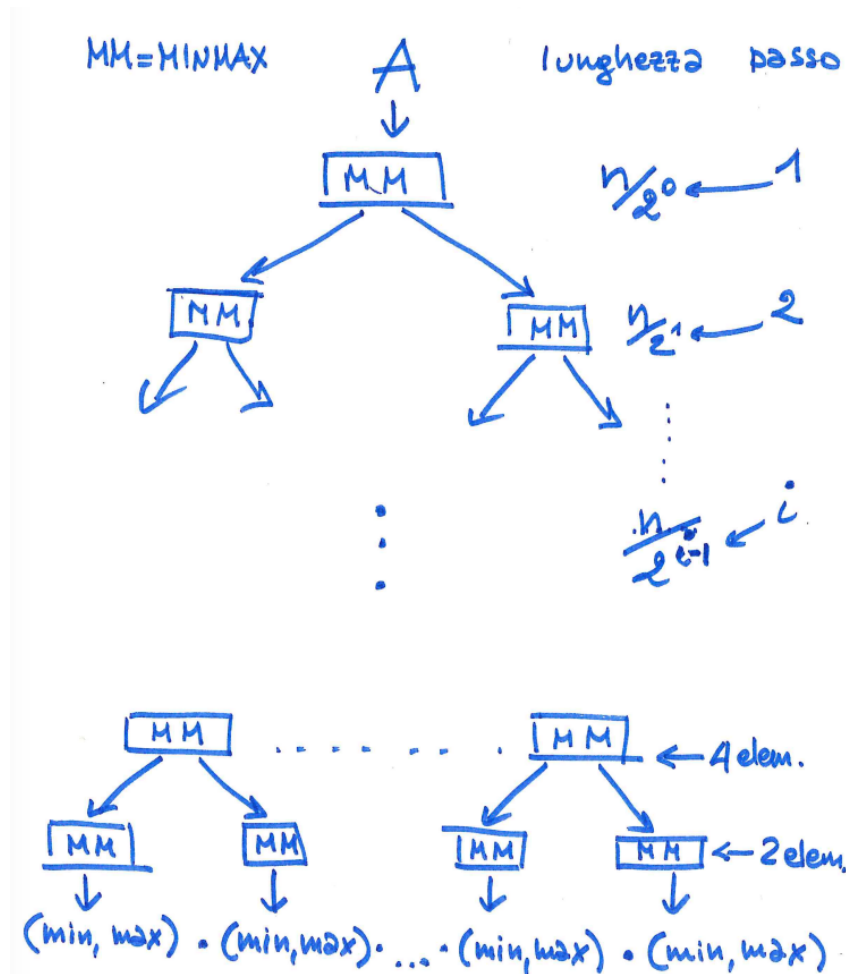
Tali proprietà suggeriscono un approccio "divide et impera" per le bitoniche:

- MINMAX suddivide il problema su n elementi in istanze più piccole: $A_{\min}$ e $A_{\max}$
- ordinando $A_{\min}$ e $A_{\max}$ la fusione (merge) avviene per concatenazione

ed ecco il bit-merge sequenziale, devo passargli un array bitonico:

```
1 bit-merge(A) {  
2   MINMAX(A)  
3   if (|A| > 2) {  
4     bit-merge(A_min)  
5     bit-merge(A_max)  
6   }  
7   return A  
8 }
```

Arriviamo all'implementazione parallela, prima abbiamo effettuato la fase delle chiamate ricorsive e poi della concatenazione, chiamiamo il modulo MinMax MM:



L'albero è di profondità logaritmica, quindi il tempo che salta fuori è $5 \log n$. Ogni passo richiede $n/2$ processori.

Vediamo l'efficienza, sopra mettiamo il tempo sequenziale dei qualunque algoritmo che ordina una sequenza generica e sotto numero di processori per tempo parallelo, il risultato è una costante diversa da zero:

$$E = \frac{n \log n}{\frac{n}{2} 5 \log n}$$

Rimane da vedere il BIT-SORT, che è un algoritmo per sequenze generiche, si ispira al merge-sort ma utilizza come funzione di fusione il bit-merge.

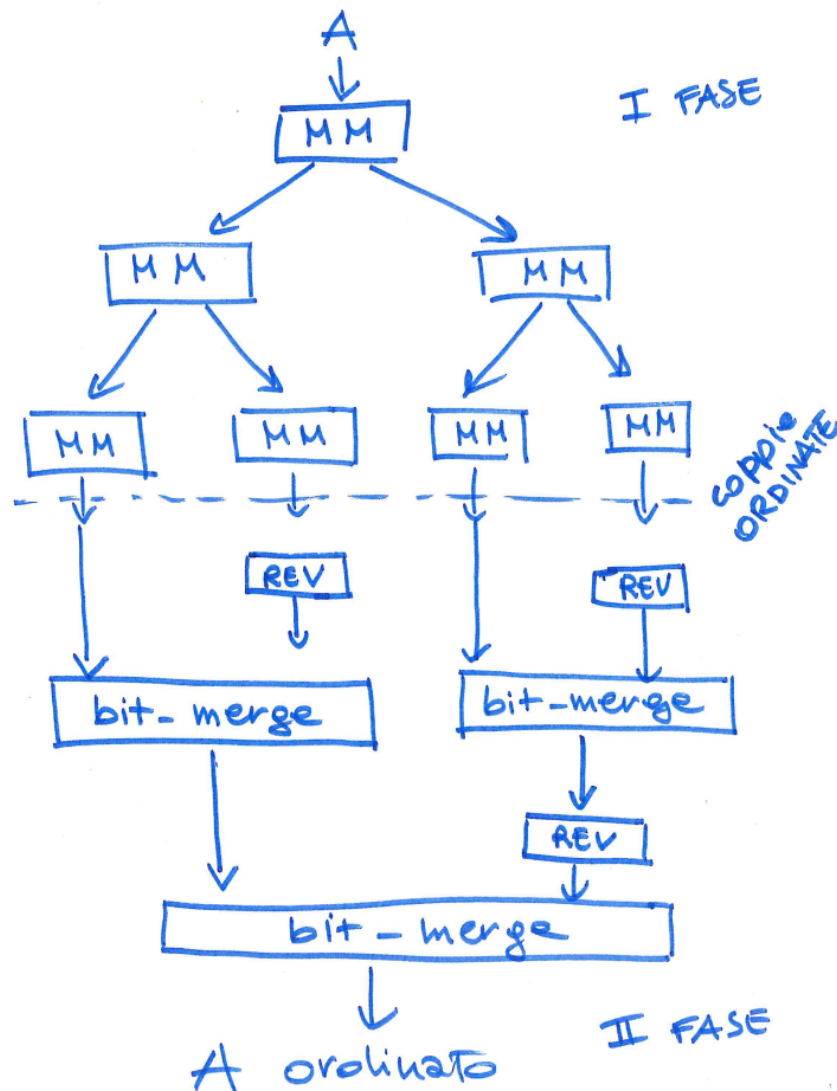
```

1 bit-sort(A){
2   MINMAX(A)
3   if (|A| > 2){
4     bit-sort(Amin)
5     bit-sort(Amax)
6     bit-merge(Amin . REV(Amax)) //unimodale -> bitonica
7   }
8   return A
9 }

```

Dopo le chiamate MINMAX si ottengono le due sequenze crescenti, che poi insieme all'operazione REVERSE si trasformano in una sequenza bitonica.

Andiamo a vedere l'implementazione parallela di bit-sort:



Valutazione:

- É un algoritmo EREW
- Tempo:
 - La prima fase se i è l'ultimo passo mi salta fuori che $i = \log n$
 - La seconda fase è la profondità dell'albero moltiplicato per REV e Bit-Merge:
 $\log n - 1 \cdot \log n = O(\log^2 n)$
- Processori: entrambe le fase concordano con $\frac{n}{2}$ processori

Ha un efficienza che tende a zero, ma lentamente, tanto lentamente che è preferibile su istanze numeriche molto piccole:

$$E = \frac{n \log n}{\frac{n}{2} 5 \log^2 n} \rightarrow \frac{\alpha}{\log n} \rightarrow 0$$

2.8.2 Osservazioni

Concludiamo con alcune osservazioni che mettono a confronto algoritmi sequenziali e paralleli. Non sempre un buon algoritmo sequenziale diventa un buon algoritmo parallelo, ad esempio il merge-sort. Ma la stessa cosa vale se abbiamo un buon algoritmo parallelo e renderlo sequenziale, vediamo i conti per valutare il tempo di bit-sort eseguito in sequenziale:

$$\begin{aligned} & \text{- Bit-merge} \\ & t_{bm}(n) = \begin{cases} O(1) & n=2 \\ 2 t_{bm}(\frac{n}{2}) + O(n) & n>2 \end{cases} \\ & t_{bm}(n) = O(n \log n) \\ & \text{- Bit-sort} \\ & t_{bs}(n) = \begin{cases} O(1) & n=2 \\ 2 t_{bs}(\frac{n}{2}) + O(n \log n) & n>2 \end{cases} \quad \begin{array}{l} \swarrow \text{bit-merge} \end{array} \\ & t_{bs}(n) = O(n \log^2 n) \end{aligned}$$

2.9 Tecnica del ciclo Euleriano

Serve per gestire strutture dati dinamiche, come gli alberi. Parliamo di grafi diretti descritti come due insiemi di vertici V e archi E , abbiamo la notazione $(v, w) \in E$ in cui indichiamo il nodo sorgente v e il nodo pozzo w . Un cammino è una sequenza di archi tale che il nodo finale coincide con il nodo iniziale e_{i+1} . Parliamo di cicli quando abbiamo un cammino tale che il nodo finale coincide con il nodo iniziale di e_i .

Parliamo di ciclo Euleriano quando ogni arco in E compare una e una sola volta. Un grafo si dice Euleriano se è possibile ottenere dal grafo un ciclo Euleriano.

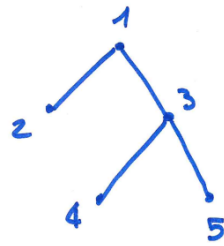
Per capire se un grafo è un ciclo Euleriano ho bisogno di introdurre la notazione, il numero di archi entranti in v e il numero di archi uscenti da v :

Vogliamo sfruttare la tecnica del ciclo Euleriano che ci permette di trasformare le nostre strutture dati in grafi che contengono cicli Euleriani. Questa tecnica è usata per costruire algoritmi paralleli efficienti che gestiscono strutture dinamiche come gli alberi binari.

Voglio gestire nella memoria condivisa la codifica dell'albero binario in termini di tabella, per farlo etichetto ogni nodo dell'albero con un numero, poi utilizzo una tabella in cui ogni riga è dedicata ad un nodo ed infine le informazioni che scrivo sul nodo sono il figlio sinistro e destro e il padre (la radice ha padre 0, le foglie hanno figli 0,0):

Ci sono molti problemi che usano gli alberi binari come ricerca, dizionari e query. L'operazione fondamentale in tali problema è la navigazione dell'albero sia per la manutenzione (modifica/cancellazione e inserimento). È possibile operare su queste strutture dati in parallelo con algoritmi efficienti? sì, si può fare con le liste, abbiamo già visto il concetto

ALBERI BINARI



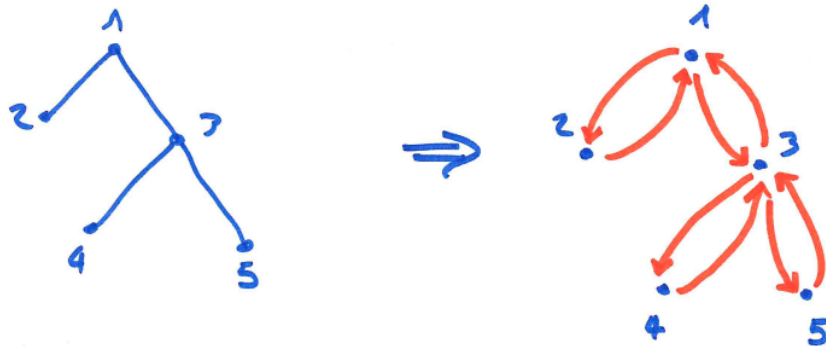
→ = foglia

v	$sin(v)$	$des(v)$	$pad(v)$
RADICE 1	2	3	0
→ 2	0	0	1
NODO INTERNO 3	4	5	1
→ 4	0	0	3
→ 5	0	0	3

di lista con l'algoritmo di Kogge-Stone. Sostanzialmente dobbiamo definire un vettore S di successori, gli elementi sono i vettori dell'albero.

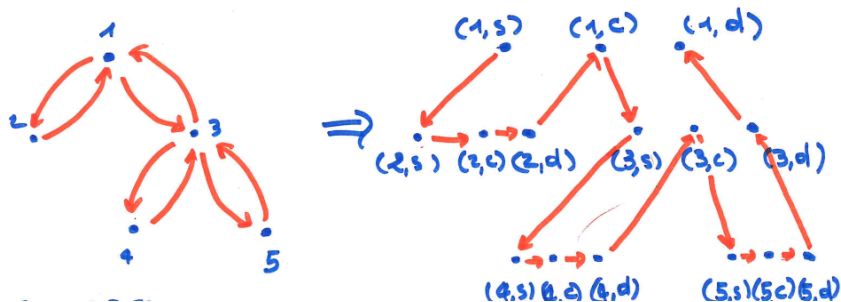
2.9.1 Primo passo

Il primo passo è associare all'albero un ciclo Euleriano, sostituisco ad ogni arco due archi, uno che scende e uno che sale. In questo modo navigo l'albero seguendo il ciclo Euleriano.



2.9.2 Secondo passo

A questo punto decido di trasformare questo grafo per definire un cammino Euleriano, parto con il grafo ottenuto dall'albero e faccio un'operazione sui nodi del grafo, ogni nodo v viene espanso in tre nodi di sinistra, di destra e di centro. Dopo di che metto degli archi che mi permettono di navigare questo grafo toccando tutti i vertici.



2.9.3 Terzo passo

Devo utilizzare un vettore $S((v, x))$ che mi costruisce una lista a partire dal grafo di prima. Per ogni nodo di x ho il suo successore.

v	$\sin(v)$	$\text{des}(v)$	$\text{pad}(v)$	
1	...		0	
2				
3	4	5	1	← nodo interno
4	0	0	3	} foglie
5	0	0	3	

Se il nodo foglia è figlio sinistro va al nodo padre con l'etichetta centro, se è di destra va al nodo padre con etichetta destra.

$$\begin{aligned}
 S[(v, s)] &= (v, c) \\
 S[(v, c)] &= (v, d) \\
 S[(v, d)] &= \begin{cases} (\text{pad}(v), c) & \text{se } v = \sin(\text{pad}(v)) \\ (\text{pad}(v), d) & \text{se } v = \text{des}(\text{pad}(v)) \end{cases}
 \end{aligned}$$

$$\begin{aligned}
 S[(v, s)] &= (\sin(v), s) \\
 S[(v, c)] &= (\text{des}(v), s) \\
 S[(v, d)] &= \begin{cases} (\text{pad}(v), c) & \text{se } v = \sin(\text{pad}(v)) \\ (\text{pad}(v), d) & \text{se } v = \text{des}(\text{pad}(v)) \end{cases}
 \end{aligned}$$

Vediamo un algoritmo parallelo per trasformare S . Si può pensare che la tabella è in memoria e ogni processore si prende una riga della tabella (ogni vertice) e costruisce il successore sinistro, centrale e destro. Per fare questo andrà a prendere la riga della tabella. Per stabilire il successivo destro deve vedere il padre di v , questo provoca delle letture concorrenti (perchè va a leggere sulla riga di un altro processore). In realtà si può eliminare la concorrenza facendo in modo che un nodo legga a sinistra e uno a destra. Quindi abbiamo un algoritmo EREW che usa $p(n) = n$ con $T(n, p(n)) = O(1)$, usando Wyllie (raggruppiamo i nodi del grafo in gruppi di $\log n$) come sempre otteniamo $p = \frac{n}{\log n}$ e $T = \log n$.

Usiamo l'array S per risolvere questi problemi:

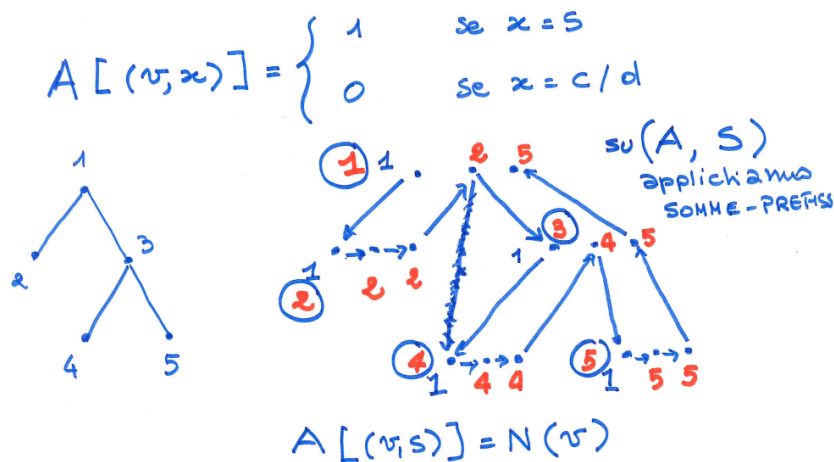
- Visitare l'albero in preordine: significa visitare la radice, il figlio di sinistra e il figlio di destra. Visita i nodi dall'alto verso il basso da sinistra verso destra.
- Calcolare la profondità dei nodi

Useremo questa notazione:

- $N(v)$: ordine di attraversamento di v in preordine. Ad esempio $N(\text{radice}) = 1$ mentre $N(\text{foglia tutta a destra}) = n$
- $P(v)$: profondità di v nell'albero. Ad esempio $P(\text{radice}) = 1/0$ mentre $P(\text{figlio radice}) = 2/1$

2.9.4 Attraversamento dell'albero in preordine

In blu nel grafo scriviamo il valore di A solo quando è uguale a 1, quando è zero non lo scriviamo.



I passi sono:

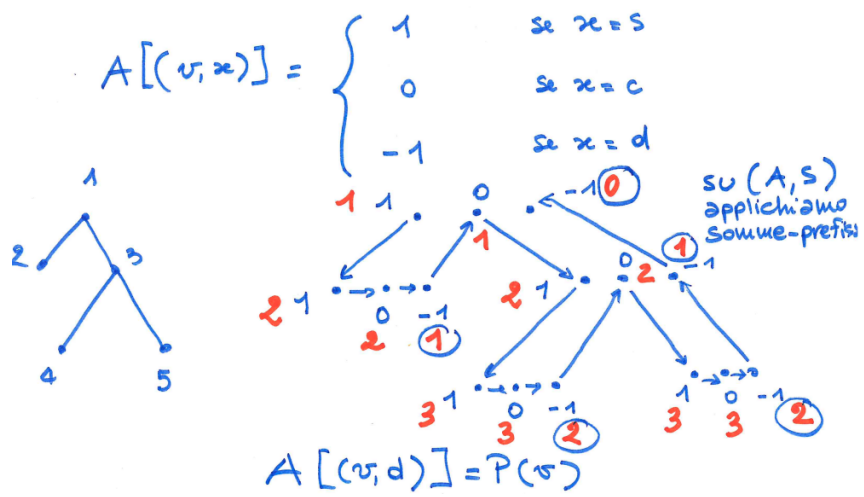
- Calcolo di A e S (successore)
- Calcolo SOMME-PREFISSE su A e S
- L'output è nel nodo $A[(v, s)]$

Otteniamo un EREW con $p(n) = \frac{n}{\log n} T(n, p(n)) = \log n E = \frac{n}{\log n \cdot \log n} \rightarrow c \neq 0$.
Troviamo così un algoritmo parallelo ottimale.

2.9.5 Calcolo della profondità dei nodi

- Calcolo A e S
- Calcolo SOMME-PREFISSE su (A, S)
- L'output è in $A[(v, s)]$

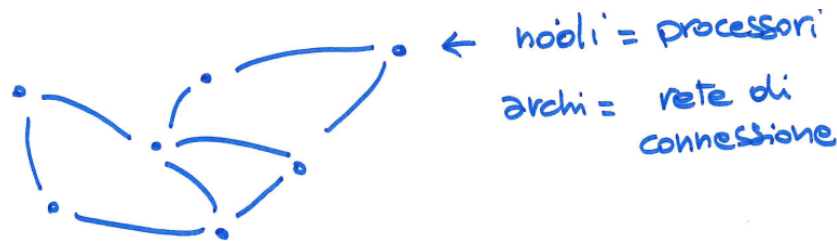
Le prestazioni sono come prima $E = \frac{n}{\log n \cdot \log n} \rightarrow c \neq 0$.



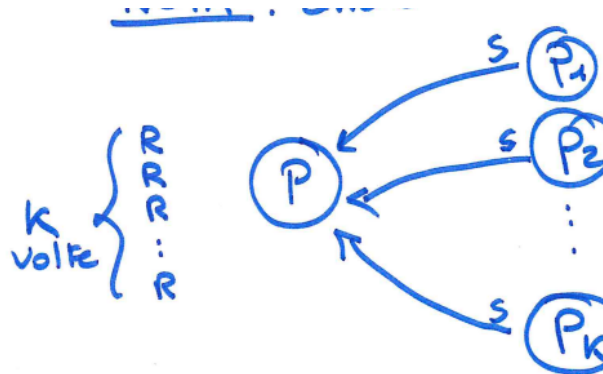
Chapter 3

Architetture Parallele a Memoria Distribuita

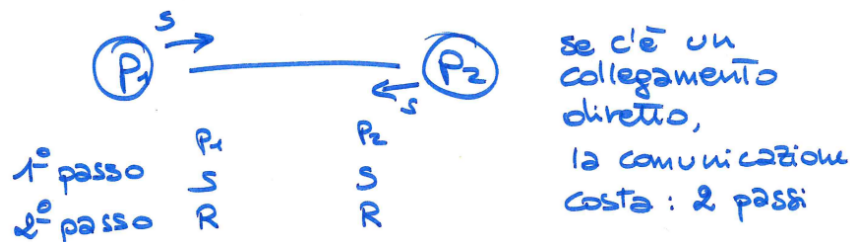
I nodi non sono solo nodi ma anche router perchè la rete è modellata come un grafo:



le send possono viaggiare in parallelo ma le receive vengono fatte per forza in sequenziale:



La comunicazione è full duplex. Abbiamo sempre un clock centrale e anche il programma è come le PRAM abbiamo i par do. Però abbiamo anche le istruzioni send e receive:



L'input viene distribuito tra i processori, perchè non c'è una memoria condivisa. L'output può essere dato da un processore dedicato oppure si legge in un certo ordine tra i vari processori.

Risorse di calcolo:

- Numero di processori: può essere la lunghezza dell'input, ma ci sono tecniche per abbassare il numero
- Tempo dato da:
 - Tempo di calcolo
 - Tempo di comunicazione che può essere più o meno rilevante

Si definiscono i seguenti PARAMETRI DI RETE:

- Grado di G: per ogni v si calcola il numero di archi incidenti su v e si ritorna il massimo. Più è alto più la comunicazione può essere buona. $\gamma = \max \rho(v) | v \in V$
- Diametro di G: per ogni coppia di vertici la distanza è il cammino minimo tra i due. Il diametro è il massimo tra tutte le distanze prese a coppie nel grafo, la massima distanza tra coppie. Valori bassi sono da preferire, ma aumentano il parametro gamma. $\delta = \max d(v, w) | v \neq w \in V$.
- Ampiezza di bisezione di G: β il minimo numero di archi in G che tolti mi dividono i nodi in circa due metà. Rappresenta la capacità di trasferire le informazioni da una parte all'altra del grafo.

Tipici problemi che studieremo:

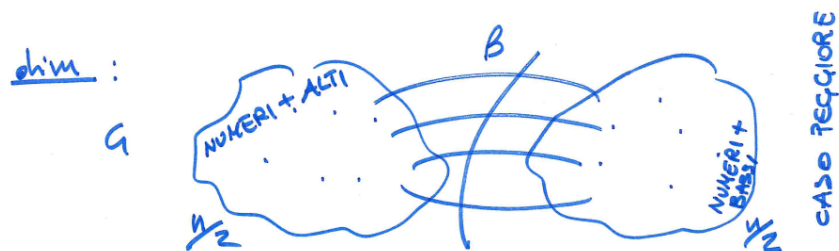
- Massimo: per effettuare confronti è necessaria la comunicazione a coppie di processori, si richiede quindi un delta massimo, in modo tale che la comunicazione tra coppie di processori avviene nel minor tempo possibile
- Ordinamento: si richiede lo spostamento di parti dell'input, questo spostamento dipende dal parametro beta, più beta sarà alto più otterremo un algoritmo efficiente

Questi problemi mettono in risalto pregi e difetti delle architetture che vedremo, array lineare e mesh.

Valgono i seguenti limiti inferiori:

- Il tempo richiesto per risolvere il massimo in G è almeno delta, perchè devo fare comunicare ogni coppia di processori
- Il tempo richiesto per risolvere l'ordinamento in G è almeno $\frac{n}{2} \cdot \frac{1}{\beta}$

L'idea è che nel caso peggiore abbiamo in metà dei processori i numeri più bassi e nell'altra metà quelli più bassi, devo quindi dover invertire tutti i numeri:



3.1 Comparatori

Per analizzare i problemi di massimo e ordinamento abbiamo bisogno di definire le istruzioni di confronto per i confrontatori o i comparatori. Un confrontatore è una linea tra i due fili e agisce in questo modo, nel filo più alto fa passare il minimo tra x e y e nella linea bassa fa passare il massimo. Esistono confrontatori che fanno il lavoro opposto. Questa operazione può essere descritta da due istruzioni.

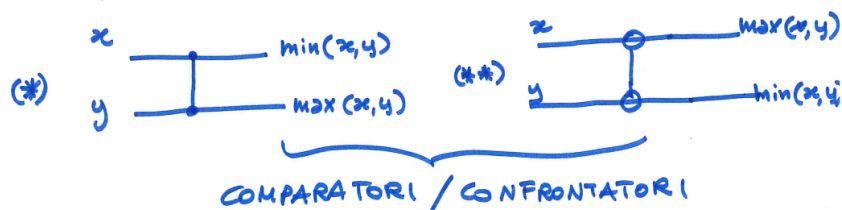
Def. le istruzioni di confronto per i confrontatori o comparatori:

(*) if ($A[i] > A[j]$) then SWAP ($A[i], A[j]$)

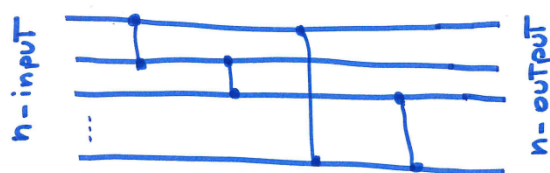
(**) if ($A[i] < A[j]$) then SWAP ($A[i], A[j]$)

con $i < j$ -

Rappresentazione di (*) e (**) con i comparatori



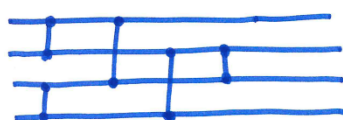
Questi comparatori possono essere messi su più fili, dobbiamo immaginarci una rete di confrontatori in cui passano gli n numeri di input. Questo è un formalismo per poter descrivere delle reti di ordinamento, gli passiamo n input da ordinare e riceviamo in output n output. Quando delle reti di confrontatori eseguono l'ordinamento costituiscono delle sorting network.



Esempi



SORTING NETWORK
Rete di confrontatori
(per 3 elementi) che ordina



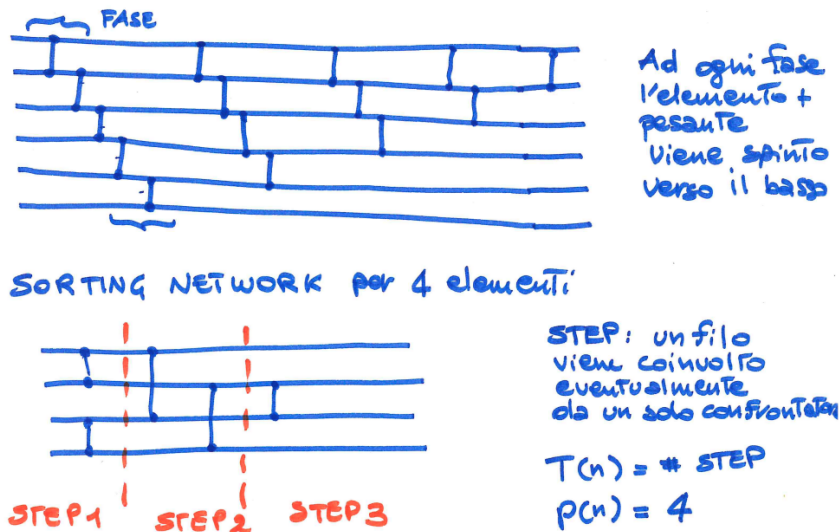
SORTING NETWORK
per 4 elementi

Nel bubble sort i confrontatori sono messi in modo sequenziale formando delle fasi, si può notare come in una fase l'elemento più pesante scende in ultima posizione.

Danno anche un'idea di come può essere parallelizzato, in una sorting network di 4 elementi possiamo dire che confrontatori che agiscono su fili diversi sono un passo parallelo. Il numero di passi mi dice in termini di tempo quando costa il processo parallelo, il numero di fili mi dice il numero di processori.

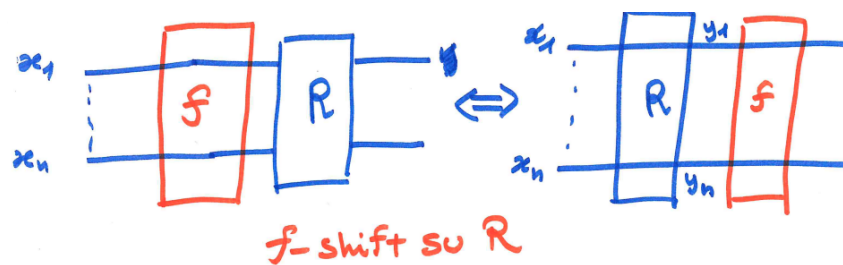
Formalmente una rete di confrontatori la indichiamo $R(x_1, \dots, x_n) = (y_1, \dots, y_n)$. Si chiama sorting network quando vale la seguente condizione per ogni n tupla di numeri naturali vale che $R(x_1, \dots, x_n) = (y_1, \dots, y_n)$ con $y_1 < \dots < y_n$. Per ogni input mi dà l'output ordinato, si chiamano anche reti di ordinamento TEST/SWAP "OBLIVIOUS" (test di confronto fissati a priori e non dipendono dall'input).

Come faccio a stabilire se una rete di confrontatori è sorting network? si usa il principio 0-1 che dice questo: se la tua rete per ogni vettore booleano la ordina allora riesce ad



ordinare ogni vettore di naturali. È una grossa semplificazione, perchè per sapere se una rete ordina i naturali lo faccio prima sui booleani.

Introduciamo l'*f-shift* su *R*. Supponiamo di avere una rete di confrontatori in cui prima applichiamo la funzione *f* e poi la rete di confrontatori. Il risultato è dato anche dalla rete in cui applichiamo *R* e poi la funzione *f*. Stiamo dicendo che il risultato non cambia se shiftiamo *R* su *f*. Affinchè questo sia vero deve essere che *f* è monotona crescente.



3.2 Array Lineari

Sono un architettura parallela a memoria distribuita. Ci sono tutti i processori collegati come se fossero su una riga.



I parametri di rete sono:

- $\gamma = 2$. Numero di archi massimo di un nodo. Ottimo per la realizzazione
- $\delta = n - 1$. Distanza massima tra due nodi nel grafo. Lower bound per MAX
- β . Lower bound per ORDINAMENTO

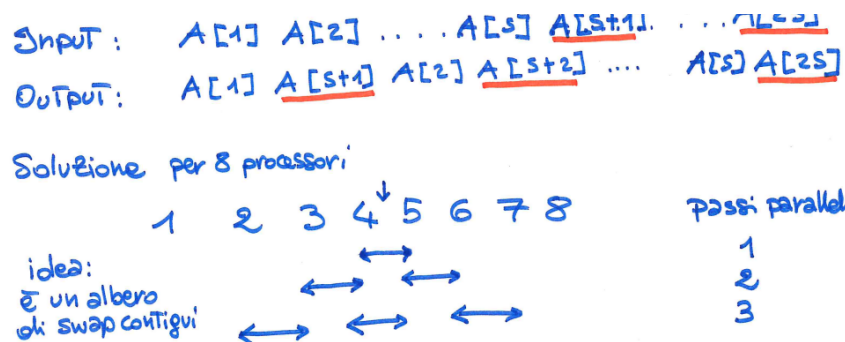
Affronteremo questi problemi:

- Shuffle
- Max
- Ordinamento

Dobbiamo pensare ad un array di processori e ognuno dei quali ha un dato. Vediamo la prima primitiva che serve al problema shuffle, lo SWAP($k, k+1$) contiguo. Abbiamo dei processori consecutivi e collegati direttamente nell'array lineare ed hanno $k+1$ e k , si chiede di scambiare i due dati. Si utilizzano le istruzioni send e receive. In un ciclo di clock fanno le due send, in quello dopo le due receive. Successivamente sovrascrivono il dato che avevano in memoria con quello arrivato. Il numero di passi è 3, è costante. Se dovessimo generalizzare lo swap tra due processori generali entrerebbe in gioco la distanza.

3.2.1 Shuffle

In input abbiamo un array ed in output vogliamo l'array mischiato, il primo elemento della seconda metà deve stare tra il primo e il secondo della prima metà. Alterniamo un elemento della prima metà con uno della seconda metà alternando gli indici. Questo problema si può risolvere facendo degli swap contigui partendo dal centro.



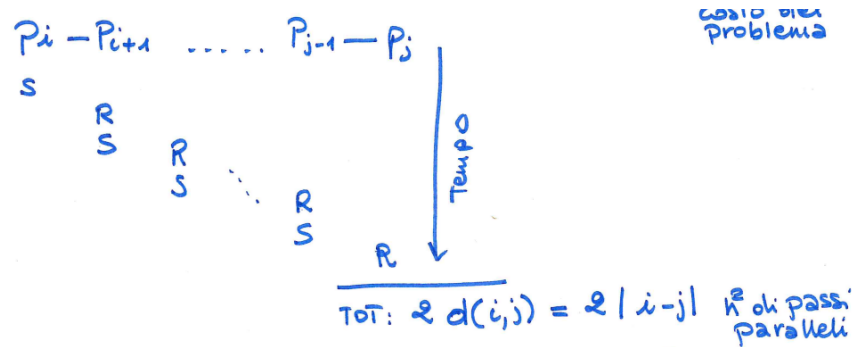
Il numero di processori dovrebbe essere n , ma in realtà il primo e l'ultimo dato non vengono mai scambiati, quindi potremmo non assegnare il processore. L'efficienza è una costante.

3.2.2 MAX

Ci sarà la richiesta del dato ad un processore j dell'array lineare, una primitiva che fa questa operazione è la SEND(i, j), ovvio che a questo punto entra in gioco la distanza tra i due processori. Ecco un algoritmo che ci permette di fare questa operazione, poi fa una send per spedire il dato a p_i+1 , fino ad arrivare al processore p_j . La prima operazione è una send e poi ad ogni ciclo di clock c'è una receive ed una send. Vediamo che il costo di una send è due volte la distanza tra i due processori.

Abbiamo n dati e vogliamo calcolare il massimo di tutti gli elementi, mettiamo il risultato nel processore con indice più alto. Abbiamo detto che il tempo per il massimo su array lineari è limitato inferiormente da n , il che rende questo algoritmo poco ragionevole perché il sequenziale è n .

L'idea è:

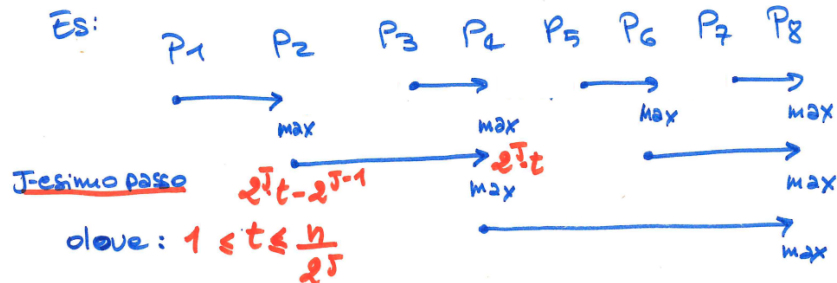


- Consideriamo l'algoritmo per SOMMATORIA delle PRAM
- Riduzione dei processori per abbassare il tempo

Opereremo in questo modo, si calcola il massimo a due a due e si va avanti così fino a che non abbiamo solo due massimi. Il j -esimo passo fa così:

- Confrontano i numeri a distanza $2^j - 1$
- Si seleziona il massimo
- Si memorizza nel processore di indice $2^j \cdot t$

Il numero di passi è $\log n$.



```

for j=1 to log n
  for k ∈ { 2^j t - 2^{j-1} | 1 ≤ t ≤ n/2^j } per do
    (send) SEND(k, k+2^{j-1})
  for k ∈ { 2^j t | 1 ≤ t ≤ n/2^j } per do
    (compare) if ( A[k] < A[k-2^{j-1}] ) Then
               A[k] = A[k-2^{j-1}]
  
```

Vediamo le prestazioni:

- send: $2 \cdot 2^{j-1}$ con $j = 1, \dots, \log n$

- compare: 2 con $j = 1, \dots, \log n$

il totale è $\sum_{j=1}^{\log n} 2 \cdot 2^{j-1} + \sum_{j=1}^{\log n} 2 = 2n - 2 + 2\log n = O(n)$ perchè è una serie nota. Il tempo parallelo in realtà è $O(n)$ e il numero di processori è n . L'efficienza tende a 0, non va bene!

Suggerimento: riduciamo i processori da n a p , in questo modo operiamo sul parametro delta.

Ho n dati e li raggruppo in n/p , in questo modo ogni processore conterrà n/p dati. Si può passare al nuovo algoritmo:

- Un processore seleziona il massimo sequenzialmente tra i suoi n/p numeri
- Si esegui il codice per MAX su p processori

vediamo le prestazioni:

- Processori: p
- Tempo: $O(n/p) + O(p)$
- efficienza: numeratore = n e al denominatore: $p(O(n/p) + O(p)) = O(n) + O(p^2) = O(n)$

Per avere E che tende ad una costanza scelgo $p/2 = n \rightarrow p = \sqrt{n}$. Di conseguenza MAX è risolto efficientemente su array lineare con $p = \sqrt{n}$ e tempo = $O(\sqrt{n})$.

3.2.3 Ordinamento

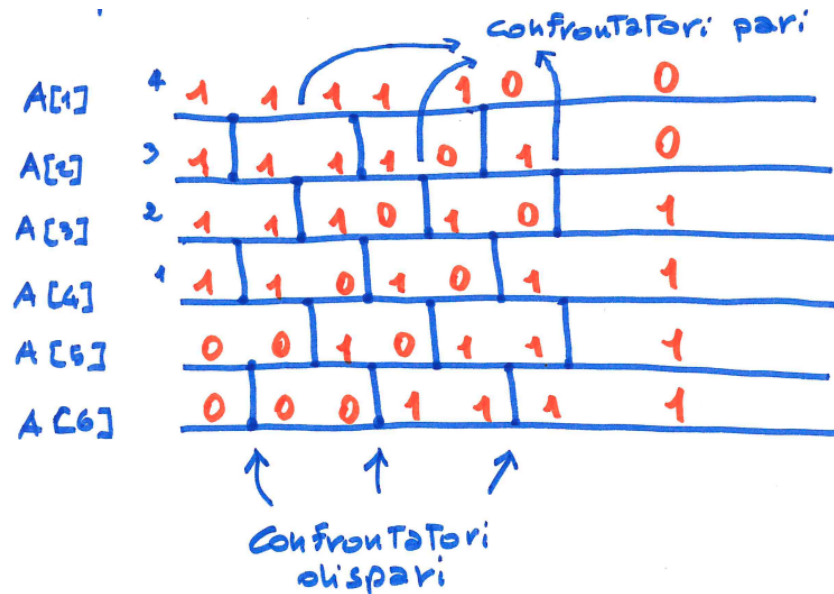
Per affrontare questo problema dobbiamo fare uno swap tra due processori non contigui. Si possono dare diverse soluzioni, una è quella di spedirsi in sequenza i valori, dopo di che faccio l'assegnamento per fare lo swap. Il problema è che facendo le send in sequenza il tempo risulta essere 4 volte la distanza. Se effettuiamo queste send in parallelo dobbiamo gestire i dati nei processori in mezzo, perchè riceveranno due dati contemporaneamente.

Dobbiamo distinguere tra due casi che sono funzioni della distanza tra due processori, pari e dispari.

Se la distanza è dispari ($2k+1$) il numero di processori sono pari, nella prima metà ho $k+1$ processori così come nella seconda. I processori che stanno al centro sono quelli di indice $k+1$ e $k+i+1$. I due dati arrivano simultaneamente ai processori, questi fanno due send simultanee perchè la connessione è full duplex. Il costo del primo caso è: $2k + 2 + 2k + 1 = 2d(i, j) + 1$ che è sostanzialmente il costo della send più l'assegnamento.

L'altra primitiva che ci serve è il MINMAX, realizza il confrontatore nelle sorting network, prende due fili e mette il minimo nel filo alto e il massimo in quello basso. Prendiamo le istruzioni dello swap e facciamo lo scambio condizionato dal confronto tra i valori.

Arriviamo all'algoritmo di ordinamento, in input abbiamo un array e in output dei processori che contengono i numeri. Usiamo una sorting network fondata su sei fili, abbiamo la prima colonna di confrontatori che si chiamano dispari, perchè si poggiano su un indice dispari (1), segue una colonna di confrontatori pari perchè il primo elemento su cui si poggiano è pari (2), la colonna di confrontatori si chiama anche round perchè tutti i



confrontatori sulla stessa colonna possono essere eseguiti in parallelo. Alterniamo quindi un round dispari ed uno pari, ci sono n round (n come numero di dati).

Per la correttezza diciamo che vale il principio 0/1.

Vediamo l'implementazione, possiamo dare anche un'implementazione sequenziale, basta eseguire sequenzialmente i confrontatori e ci costerebbe $O(n^2)$. In parallelo abbiamo n round di comparatori implementati con MINMAX:

```

1 for i = 1 to n
2   for k in {2t - (i % 2) > 1 <= t <= n/2} par do
3     MINMAX(k, k+1)

```

Abbiamo n round ed il MINMAX richiede 4 istruzioni quindi otteniamo un tempo $O(n)$. Possiamo calcolare l'efficienza: $\frac{n \log n}{n \cdot n} \rightarrow 0$. Siamo riusciti ad ottenere l'algoritmo migliore che ci poteva dare la rete. Dobbiamo abbassare i processori. Ogni processore si prende n/p dati e li ordina sequenzialmente in un tempo $O(n/p \log n/p)$.

Per l'algoritmo MERGE-SPLIT la primitiva avviene tra due processori contigui:

- il processore di sinistra spedisce i suoi n/p dati ordinati al processore di destra, questo avviene in tempo uguale al numero di dati.
- il processore di destra riceve e fonde i nuovi dati con i suoi dati. Fa il merge in tempo n/p
- Il processore di destra invia i dati più piccoli (n/p) al processore di sinistra. Operazione di SPLIT

```

1 for i = 1 to p
2   for k in {2t - (i % 2) > 1 <= t <= n/2} par do
3     MERGESPLIT(k, k+1)

```

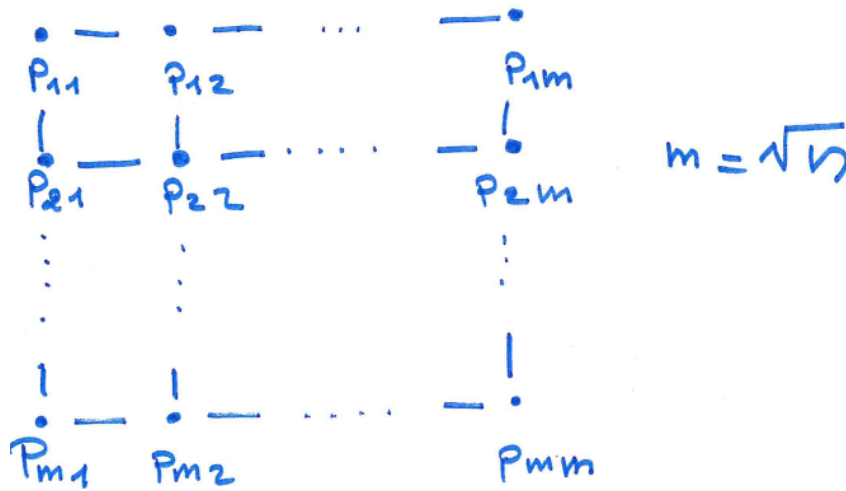
Tempo: $n/p \log n/p + p * n/p = O(n)$. Se scelgo $p = n \log n$ otteniamo un'efficienza costante diverso da zero.

Nonostante la riduzione dei processori il tempo è rimasto $O(n)$, questo si spiega in quanto la riduzione dei processori agisce sul diametro e non sull'ampiezza di bisezione beta.

3.3 Mesh

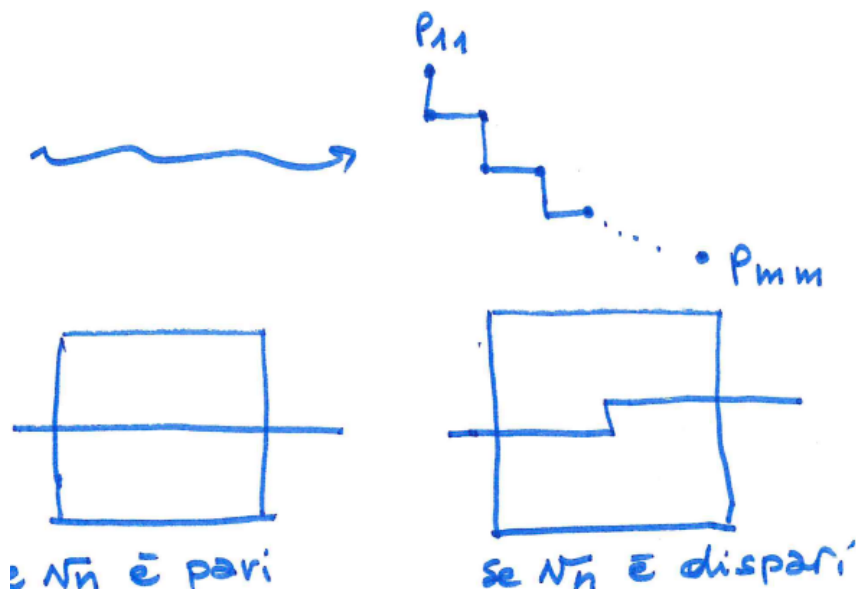
Sono le architetture utilizzate dai supercomputer, invece di avere un array lineare abbiamo un array multidimensionale. I processori sono messi per riga e colonna, così come i collegamenti.

Se n non è un quadrato perfetto prendiamo questa quantità $(\sqrt{n} + 1)^2 \leq 2n$.



Vediamo i parametri di rete:

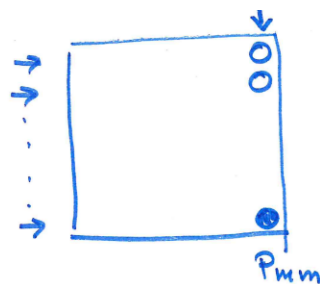
- $\rho = 4$ un processore è collegato al massimo con 4 processori
- $\delta = 2\sqrt{n}$ la distanza, i processori più distanti sono quelli in diagonale
- $\beta = \sqrt{n}$ quanti link devo togliere per dividere la griglia in due parti uguali



3.3.1 MAX

L'idea immediata è di usare la mesh come array lineare costruendo un percorso a serpente ottenendo un array lineare di n processori. Non va bene, pensiamo ad un algoritmo che sfrutta tutte le connessioni che vengono ignorate.

Un'idea è quella di fare il max con il RIGHE-COLONNA in cui ogni riga è vista come un array lineare di $\sqrt{n}$ processore e l'ultima colonna è un altro array lineare. Sposto i massimi di ogni riga sull'ultima colonna e poi calcolo il massimo tra tutti i massimi:



ogni riga è vista
come un array lineare
di $\sqrt{n}$ proces.

l'ultima colonna è
un altro array lineare di
 $\sqrt{n}$ proces.

```

1 for i = 1 to radice(n) par do
2   MAX(pil, ..., pim)
3
4 MAX(p1m, ..., pmm)

```

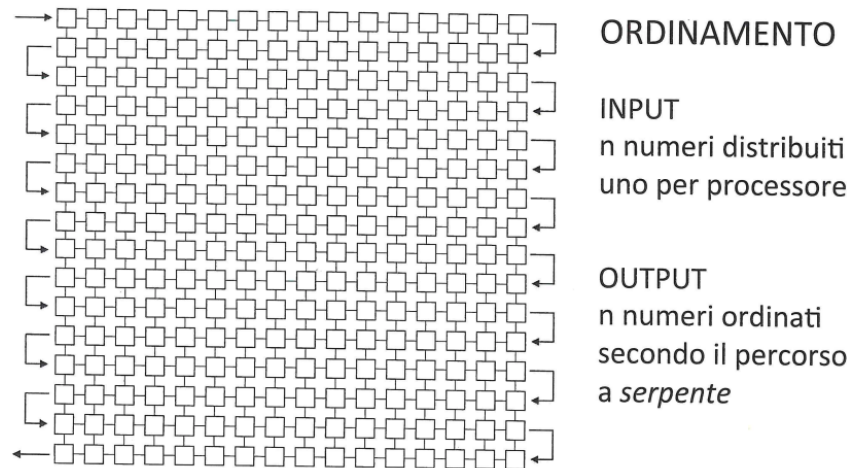
costa un $O(\sqrt{n})$. Così otteniamo un'efficienza che tende a zero, non va bene, quindi operiamo una riduzione dei processori da n a p .

- Ogni processore calcola il massimo tra n/p dati in tempo $O(n/p)$
- si attiva l'algoritmo di prima sulla griglia $\sqrt{px}\sqrt{p}$. Quindi il tempo è $O(\sqrt{p})$

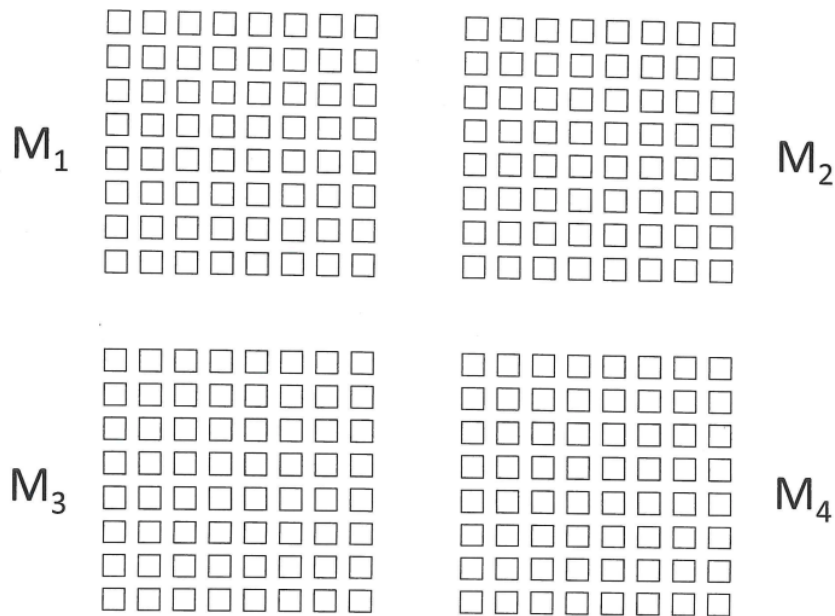
Otteniamo così un tempo totale $\frac{n}{p} + \sqrt{p}$.

3.3.2 Ordinamento

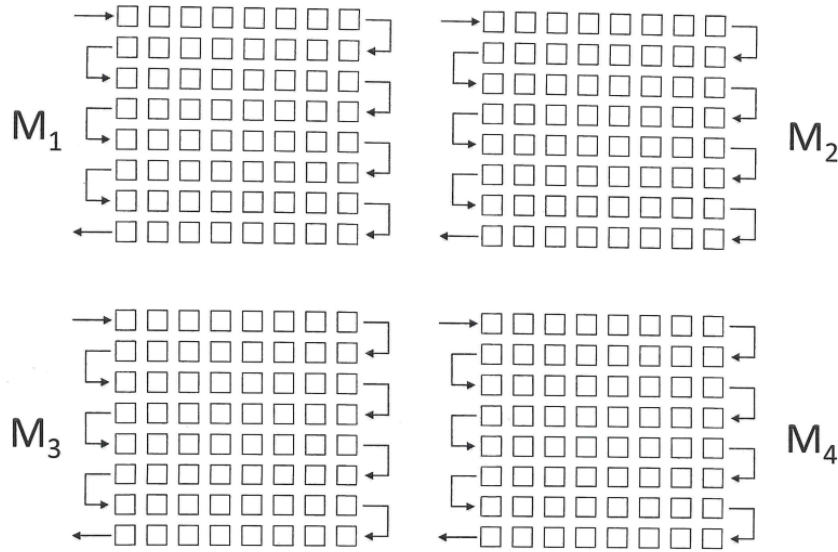
I dati in input sono distribuiti uno per processore, se l'indice della riga è dispari i numeri vengono distribuiti da sinistra a destra, il contrario se l'indice è pari. Ci aspettiamo che l'output sia ordinato secondo il percorso a serpente.



L'algoritmo si chiama LS3 che applica la tecnica divide et impera. Chiamiamo M l'intero quadrato dei processori la fase divide divide il quadrato in 4 quadrati di dimensione la metà del lato di m :



Con le chiamate ricorsive si riesce ad ordinare ogni metà. La terza fase è la fusione, una voglia che arrivano 4 matrici ordinate:



la terza fase prende queste quattro matrici ordinate e restituisce la matrice M completamente ordinata.

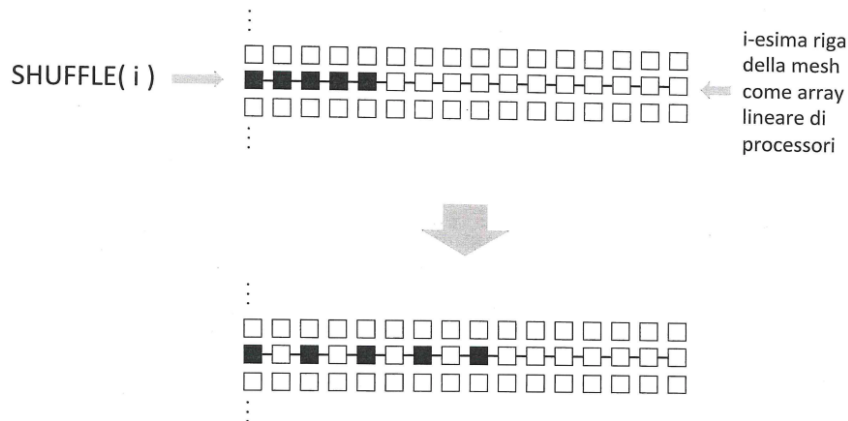
Vediamo il codice:

```

1 LS3sort(M)
2   if (|M| == 1)
3     return M
4   else
5     LS3Sort(M1) //in parallelo
6     LS3Sort(M2)
7     LS3Sort(M3)
8     LS3Sort(M4)
9     LS3Merge(M1, M2, M3, M4)

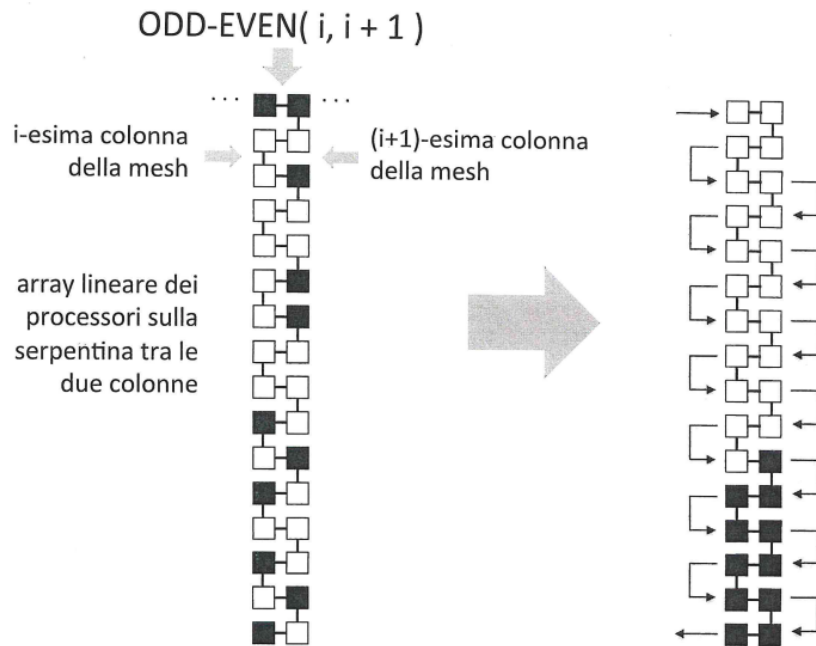
```

In Merge si fanno due operazioni shuffle ed odd even. La spiegazione di queste procedure le diamo visivamente con una mesh a cui sono stati affidati input binari (bianco 0 nero 1). Si fa shuffle di ogni riga, prende le 4 matrici e le considera come una sola, lo shuffle lo fa sulla riga di m. Considera la riga come un array lineare di processori, e abbiamo già visto come si fa lo shuffle su un array lineare:



Dopo di che si mette a fare l'ordinamento odd-even tra due colonne successive, vuole

vedere le due colonne come un array lineare, per farlo deve mettere le connessioni, anche questa volta le connessioni sono messe a serpente:



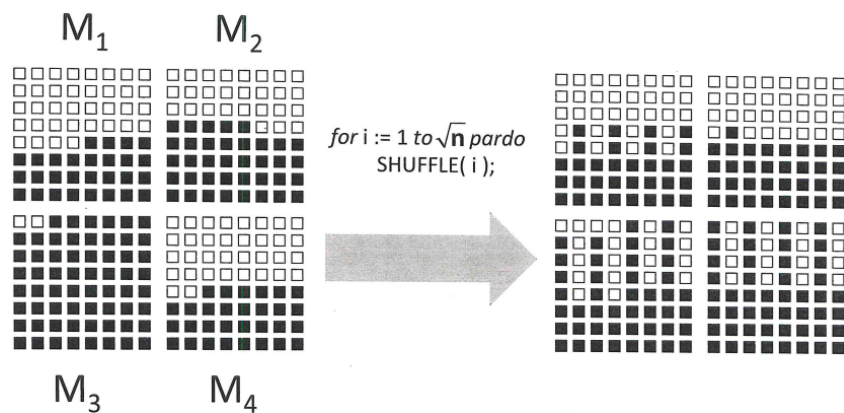
Vediamo il codice del merge:

```

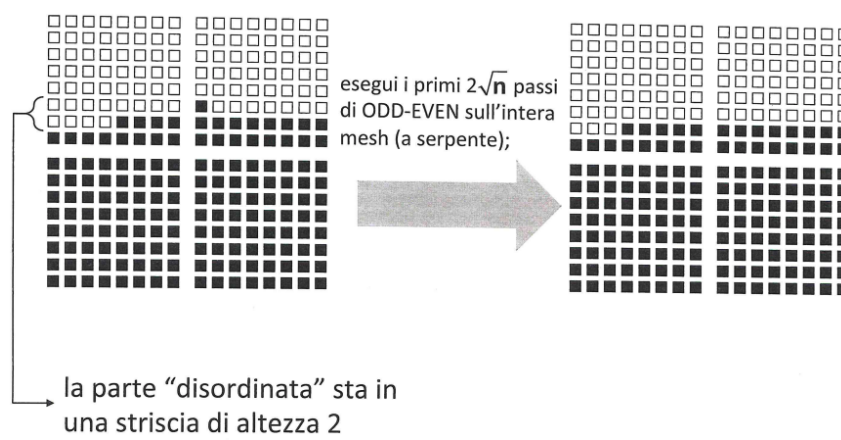
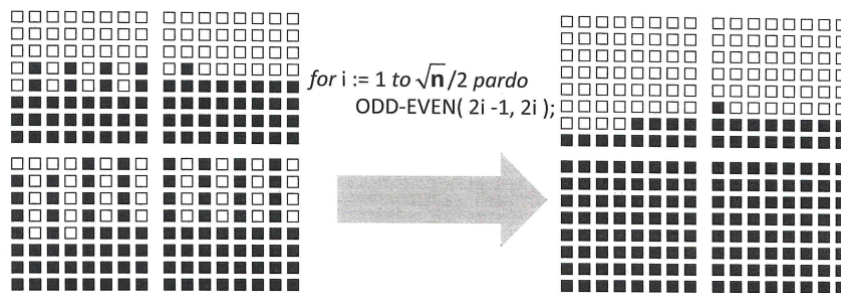
1 LS3merge(M1, M2, M3, M4)
2   for i := to radice(n) par do
3     SHUFFLE(i)
4
5   for i := to radice(n/2) par do
6     ODD-EVEN(2i - 1, 2i)
7
8   esegui i primi passi 2radice(n) passi di
9   ODD-EVEN sull'intera mesh

```

Vediamo un esempio di come funziona questo algoritmo con in input un array binario:



Ed infine calcoliamo il tempo:



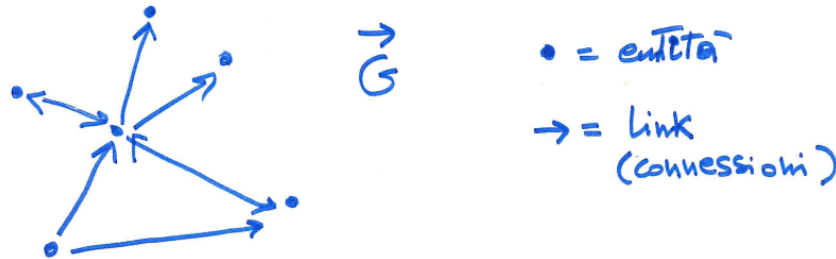
	Tempo
for $i := 1$ to $\sqrt{n}$ pardo SHUFFLE(i);	$O(\sqrt{n})$
for $i := 1$ to $\sqrt{n}/2$ pardo ODD-EVEN($2i - 1, 2i$);	$O(\sqrt{n})$
esegui i primi $2\sqrt{n}$ passi di ODD-EVEN sull'intera mesh (a serpente);	$O(\sqrt{n})$
$T_{\text{merge}}(n) = h\sqrt{n}$	

Chapter 4

Ambiente di calcolo distribuito

Abbiamo un'architettura distribuita e vogliamo risolvere problemi di comunicazione nell'ambiente distribuito, in particolare vedremo problemi di sincronizzazione dei processi e altri problemi del leader e dell'instradamento.

L'ambiente è modellato come un grafo, i vertici sono le entità e i link sono le connessioni, non è detto che le connessioni siano tutte full duplex.



Parliamo delle proprietà delle entità, che possiede:

- memoria locale
- capacità di calcolo
- capacità di comunicazione
- clock locale all'entità. Non c'è una sincronizzazione tra le entità

Esempi di entità sono : processori, processi, sensori, switch...

Nella memoria locale ci sono i registri su cui possono compiere le operazioni:

- registri di input: $\text{valore}(x)$ è l'input dell'entità x
- registro di stato: $\text{stato}(x)$ è lo stato dell'entità x . Lo stato fa capire all'entità la situazione in cui si trova, in base allo stato fa operazioni invece di altre. Lo stato è cambiato localmente dalla stessa entità x .

Per il clock locale è possibile settare o resettare una sveglia.

Proprietà entità:

- Sono reattive, solo quando accade un evento compiono un'azione:
 - Eventi:
 - * Interni al sistema: sono ricezione di messaggi o la sveglia
 - * Esterni al sistema: impulso spontaneo. Quando un'entità riceve un impulso spontaneo facciamo partire l'algoritmo distribuito
 - Azione: sono una sequenza finita di operazioni indivisibile. Una volta che inizia l'azione la porta a termine, non è possibile interrompere o cambiare azione. Un esempio di azione è l'azione nulla: nil.

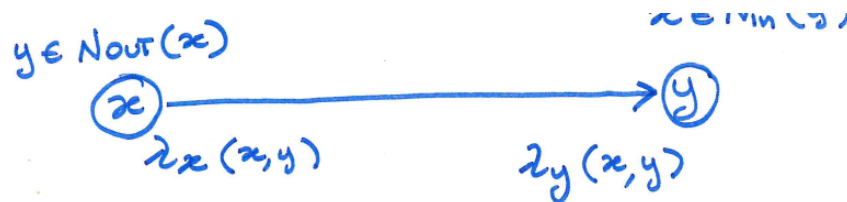
Le entità seguono delle regole, una regola è un oggetto della forma stato x evento, ad ogni coppia stato, evento segue un'azione specifica.

Se x è un'entità indichiamo con $B(x)$ l'insieme delle regole a cui è soggetta x . $B(x)$ deve essere completo e non ambiguo. $B(x)$ è anche chiamato il codice di x .

Se E è l'insieme di tutte le entità indichiamo come $B(E)$ il comportamento del sistema che è ottenuto facendo l'unione di tutti i $B(x)$ delle entità del sistema. È importante che il comportamento del sistema $B(E)$ sia omogeneo, significa che per ogni entità x e y sia $B(x) = B(y)$. Vogliamo un unico codice, quando è omogeneo lo chiamiamo protocollo per E oppure algoritmo distribuito per E . È sempre possibile ottenere un protocollo omogeneo per un sistema. Per fare questa operazione si usa un registro $Ruolo(x)$ all'interno dell'entità, facciamo in modo che questo registro sia diverso per diverse entità che hanno la stessa coppia (stato, evento).

Veniamo alle proprietà della rete:

- La comunicazione avviene usando una etichettatura sui link. Per l'entità x l'etichettatura è denotata con λx . Dato che x si trova in un grafo diretto G indichiamo con $Nin(x)$ i vicini di ingresso ad x (tutti i nodi del grafo che hanno un link in ingresso a x) e con $Nout(x)$ i vicini in uscita di x (tutti i nodi raggiunti da x tramite dei link).
- Assiomi:
 - Ritardo finito di comunicazione: quando spedisco un messaggio, in assenza di errori, prima o poi il messaggio arriverà a destinazione
 - Orientamento locale: ogni entità riesce a distinguere tra i suoi vicini Nin e $Nout$ grazie alla conoscenza della funzione λx



Anche in questo caso abbiamo i parametri di rete:

- Numero di entità = n
- Numero di link = m
- Diametro della rete = d

Abbiamo anche delle restrizioni sulla rete, sono delle proprietà positive sulla rete su cui noi facciamo affidamento. Restrizioni sulla comunicazione:

- Link Bidirezionali: supponiamo sempre che i link tra le entità sono sempre full duplex. Vediamo G come grafo indiretto. $\forall x \rightarrow Nin(x) = Nout(x) \rightarrow N(x)$
- Ordinamento dei messaggi: i messaggi sullo stesso link vengono prelevati con la politica FIFO

Restrizioni sull'affidabilità della rete:

- Rilevazione degli errori: ogni entità si accorge che un'entità non funziona o che un link si è guastato.

- Affidabilità parziale: non ci saranno errori in futuro, ma ci potrebbero essere stati errori in passato
- Affidabilità totale: non ci sono stati errori e non ce ne saranno in futuro

Restrizioni sulla topologia della rete:

- Connettività del grafo: vogliamo che ogni entità della rete possa essere raggiunta da ogni altra entità. Si parla di grafo fortemente connesso (diretto) o connesso (indiretto).

Restrizioni sul tempo:

- Tempi di comunicazione unitari: i messaggi arrivano dopo un clock
- Clock sincronizzati

Se mettessimo restrizioni di questo tipo avremmo un'architettura parallela. A volte potremmo considerare queste restrizioni quando facciamo il calcolo delle prestazioni IDE-ALI del codice distribuito.

Le misure di complessità sono:

- Tempo; l'intervallo tra la prima entità che si attiva e l'ultima che termina. Questo può essere un problema perchè esecuzioni diverse dello stesso codice distribuito possono portare a tempi diversi. Questo è un problema che si risolve considerando il tempo ideale.
- Quantità di comunicazione:
 - Numero di messaggi spediti (quando i messaggi sono omogenei, hanno la stessa dimensione)
 - Numero di bit spediti

Il tempo distribuito viene calcolato considerando comunicazioni unitarie e i clock sincroni. Il tempo ideale si contrappone al tempo causale, che viene misurato considerando la catena di comunicazione più lunga richiesta dal codice.

Vediamo come viene definito un problema, da una tripla con due predicati:

- Pinit: configurazione del sistema all'inizio (come devo essere i registri delle entità all'inizio)
- Pfinal: configurazione del sistema alla fine
- R: restrizioni del sistema

$$P = \langle P_{init}, P_{final}, R \rangle$$

Ad esempio vediamo il P: Broadcasting. Definiamo:

- Pinit: un'entità detiene I(info). $\exists x \in E | valore(x) = I$. Un'entità detiene un'informazione e tutte le altre no.

- Pfinal: tutte le entità posseggono I. $\forall x \in E \rightarrow valore(x) = I$

Dopo di che si definiscono le restrizioni:

- link bidirezionali BL
- affidabilità totale TR
- connettività CN
- unico iniziatore UI. Diciamo che solo l'entità che ha I faccia partire l'algoritmo.

BL, TR e CN si indicano con la sigla R, mentre la restrizione UI si indica con I, quindi il nostro insieme di restrizioni è denotato con RI.

Il protocollo ricapitolando è un insieme di regole della forma: stato x evento $\rightarrow$ azione.

- $stato_t(x)$: stato dell'entità x al tempo t
- evento: impulso spontaneo, sveglia, ricezione di un messaggio
- azione: mini programma indivisibile

L'esecuzione di un protocollo provoca la trasformazione del sistema stessa, è una sequenza di configurazioni successive del sistema. Vediamo la definizione di una configurazione di un sistema:

- $\Sigma(t)$: il contenuto dei registri delle entità al tempo t
- Futuro(t): eventi già generati al tempo t ma non sono ancora stati processati

Quindi la configurazione la indichiamo come: $C(t) = (\Sigma(t), Futuro(t))$.

Ad esempio la configurazione 0 ha come sigma zero i registri inizializzati e come futuro zero l'impulso spontaneo che non è ancora stato realizzato.

Quando una configurazione C soddisfa un predicato P scriveremo che $C \in P$.

Dobbiamo definire come un protocollo risolve un problema.

4.1 BCAST

Abbiamo già visto la definizione del protocollo broadcast, useremo due stati:

- Sinit: stati delle entità in C(0)
- Sterm: stati delle entità in C(f)

Vediamo un protocollo per broadcast:

- S = iniziatore, inattivo
- Sinit = iniziatore, attivo
- Sfinal = inattivo

Iniziatore:

```
1 impulso spontaneo{
2   send(M) to N(x)
3   become inattivo
4 }
```

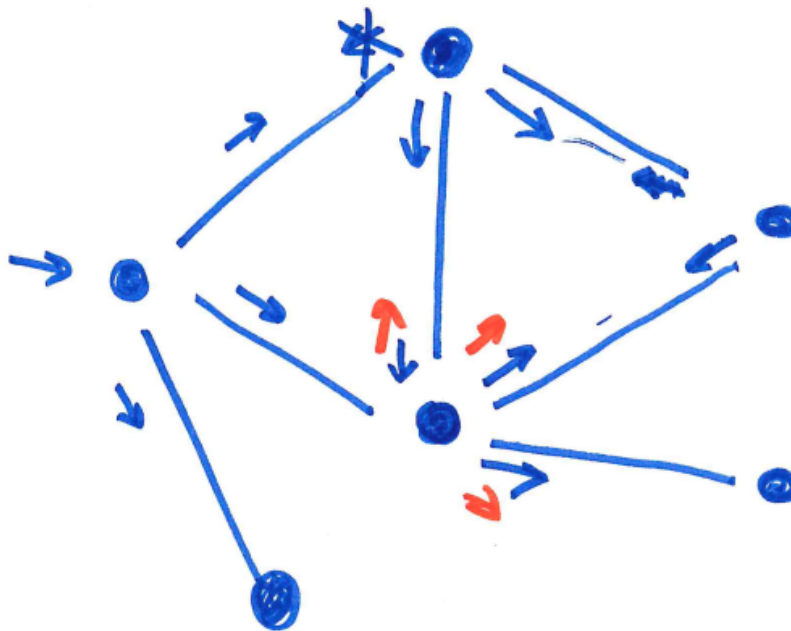
Inattivo:

```
1 ricezione(M){
2   processa(M)
3   send(M) to N(x)
4 }
```

Dobbiamo immaginarci un messaggio che viaggia in rete così composto: $M = (t, o, d, I)$

- t: tipologia del messaggio (codice)
- o: origine
- d: destinatario
- I: informazione I da prelevare e mettere nel registro valore(x)

In questa versione c'è un problema, è vero che ogni entità riceve l'informazione quando si sveglia ma il protocollo non termina mai:



La risposta a questo problema è stabilire più stati per ogni entità:

- Sstart: sono un sottoinsieme degli stati Sinit e sono quelli che fanno iniziare il protocollo

- S_{final} : sono un sottoinsieme degli stati S_{term} per cui la sola azione possibile è quella nulla

Grazie all'uso dello stato finito il protocollo riesce a terminare. Formalmente la soluzione per un protocollo è la sua correttezza: $\forall C(0) \in P_{init} \exists t' \forall t > t' c(t) \in P_{final}$ e che termini: $\forall x \in E \rightarrow stato_t(x) \in S_{final}$.

Vediamo un secondo protocollo BCAST che si chiama flooding:

- $S = \{\text{iniziatore, inattivo, finito}\}$
- $S_{start} = \{\text{iniziatore}\}$
- $S_{final} = \{\text{finito}\}$

Iniziatore:

```

1 impulso spontaneo {
2   send(M) to N(x)
3   become finito
4 }
```

Inattivo:

```

1 ricezione(M) {
2   processa(M)
3   send(M) to N(x) — sender
4   become finito
5 }
```

- Numero di messaggi $M[\text{Flooding}]$: $\sum_{x \in E} (N(x) - 1) + 1 = 2m - n + 1$
- $T[\text{Flooding}] \leq d$, d è il diametro della rete.

Si hanno anche dei lower bound per questo problema:

- Numero di messaggi $M[\text{Broadcast/RI}]$: $\neq m$
- $T[\text{Broadcast/RI}] \neq d$, d è il diametro della rete.

4.2 Wake-Up

Si può vedere come un problema BCAST ma in cui ci sono più di un iniziatore. Abbiamo un protocollo WFlood in cui abbiamo questi stati:

- $S = \{\text{dormiente, attivo}\}$
- $S_{init} = \{\text{dormiente}\}$
- $S_{term} = \{\text{attivo}\}$

Dormiente:

```

1 impulso spontaneo {
2   send(w) to n(x)
3   become attivo
4 }
5
6 ricezione (w){
7   send(w) to n(x) - sender
8   become attivo
9 }

```

COSTO :

- $T[WFlood] \leq d$

- $2^{m-n+1} \leq M[WFlood] \leq 2^m$

↑
1 sola entità
si attiva

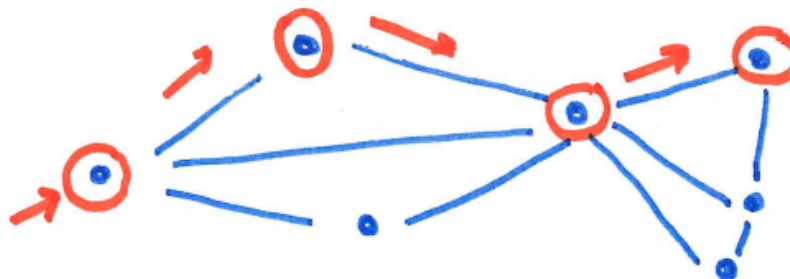
↑
tutte le entità
si attivano

4.3 Traversal

Richiede ad ogni entità di attivarsi attraverso l'arrivo di un messaggio ma sequenzialmente, cioè una dopo l'altra.

Come stati abbiamo una configurazione iniziale in cui abbia una sola entità visitata, in quella finale invece abbiamo tutte le entità visitate, ma una alla volta.

Il protocollo si chiama Depth-first traversal e compie una visita in profondità di un grafo, parte da un entità e scende al vicino dell'entità.



Il messaggio che sveglia l'entità si chiama TOKEN T, se vogliamo che si sveglino sequenzialmente le entità vogliamo che ad ogni istante di tempo viaggi un solo token. Quando un nodo riceve un token diventa visitato.

Vediamo i passi principali, un nodo che riceve T per la prima volta:

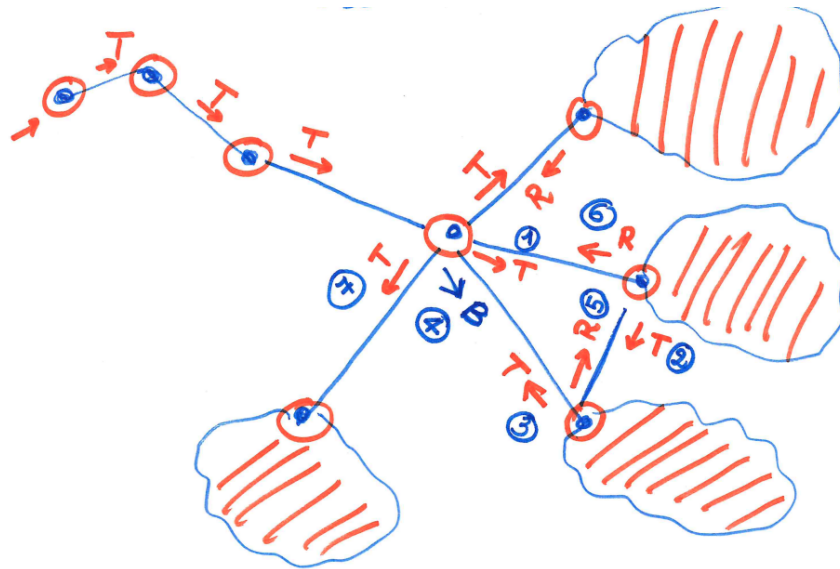
- ricorda il sender

- fa una lista dei suoi vicini non visitati
- invia T ad un vicino non visitato
- aspetta un messaggio da quest'ultima entità:
 - RETURN
 - BACK-EDGE

Vediamo cosa succede quando il vicino riceve il token:

- Se è il suo primo token ripete come prima
- altrimenti (è già stato visitato) risponde spedendo un messaggio di BACK-EDGE

Solo dopo aver finito la lista dei vicini a cui mandare il token manda un messaggio RETURN al sender.



Riassumendo abbiamo tre tipi di messaggi:

- T token
- B Back-edge
- R Return

Vediamo il codice del protocollo DF-TRAVERSAL, partiamo dagli stati:

- $S = \{ \text{initiator, idle, visited, done} \}$
- $S_{\text{init}} = \{ \text{initiator, idle} \}$
- $S_{\text{term}} = \{ \text{done} \}$

Le restrizioni sono le solite RI (bidirectional link, total reliability, connectivity, e single initiator).

Initiator:

```
1 Spontaneously{
2     initiator = True //per distinguere se sono initiator o no
3     Unvisited = N(x) //inizialmente tutti i vicini in una lista
4     Visit
5 }
```

Idle:

```
1 Receiving(T) {
2     entry = sender
3     unvisited = N(x) - sender
4     initiator = false
5     visit
6 }
```

Visit:

```
1 visit{
2     if unvisited != 0 then {
3         next = unvisited
4         send(T) to next
5         become visited
6     } else {
7         if(initiator = false) then {
8             send(Return) to entry
9             become done
10        }
11    }
12 }
```

Visited:

```
1 Receiving(Return){
2     Visit
3 }
4
5 Receiving(Back-edge){
6     Visit
7 }
8
9 Receiving(T) {
10     unvisited = unvisited - sender
11     send(Back-edge) to sender
12 }
```

Per tutte le altre coppie stato-azione il protocollo da seguire è null, non devono fare nulla.

Vediamo la complessità, se x e y sono due entità su ogni link passa un messaggio Token e un return o un back-edge, su ogni link viaggiano sempre due messaggi. L'idea è che il traversal è sequenziale perchè il token viaggia sequenzialmente. Il protocollo è

sequenziale quindi posso misurare il tempo ideale e il numero di messaggi per calcolare il tempo, trovando $2m$.

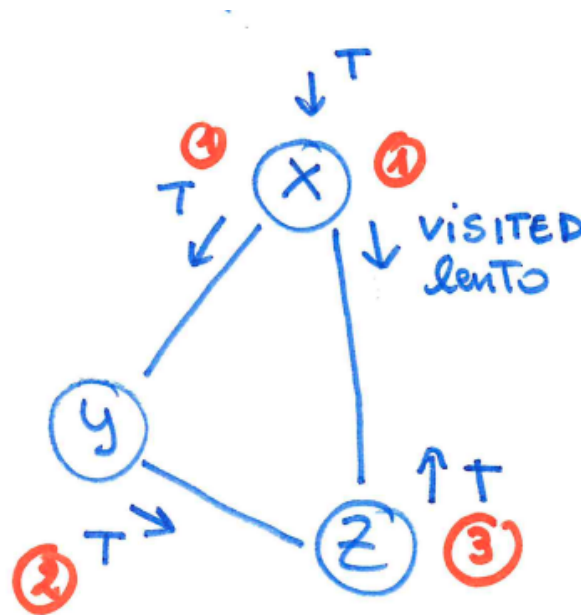
Possiamo dare dei lower bound:

- per la quantità di messaggi abbiamo m messaggi come lower bound
- per il tempo abbiamo un lower bound di $n-1$. Siccome ogni nodo deve essere visitato in sequenza

Il numero di link relazionato alle entità è compreso tra $n-1$ e n^2 . Quindi è ottimo per la quantità di messaggi, ma nel caso peggiore non per il tempo.

Il problema per il costo del tempo è che ad ogni istante di tempo viaggia un solo messaggio, una soluzione è introdurre concorrenza, magari aggiungendo una quantità opportuna di messaggi.

Il problema è la perdita di tempo inutile aspettando un back-edge, l'idea per risolvere questa cosa è che un nodo non visitato che riceve T comunica l'evento ai vicini mandando un messaggio Visited (in contemporanea con un T). I vicini che ricevono Visited aggiornano la propria lista unvisited eliminando il nodo che ha mandato visited. Non abbiamo evitato di inviare T sui back-edge, ma il protocollo è stato comunque migliorato.



Z riceve visite: capisce l'errore e spedisce T ad un'altra entità oppure invia un return e sarà questa entità che lo riceva a spedire un altro T . x riceve T : capisce che Z è visitato e lo rimuove dalla sua lista. Così abbiamo accelerato il protocollo, perchè x non ha bisogno di inviare il messaggio di back-edge dopo il token, perchè è il messaggio visited che dà la risposta a z per il messaggio inviato a x .

Nuova complessità:

$$\begin{array}{rcl}
 2n-2 & \leftarrow & T + \text{Return} \\
 2m - (n-1) & \leftarrow & \text{visited} \\
 \\
 2(m - (n-1)) & \leftarrow & \begin{array}{l} \text{errori} \\ \text{invio dei T} \\ \text{sui backedge} \end{array} \\
 \hline
 O(m) & & \begin{array}{c} \xrightarrow{T} \\ T \leftarrow \end{array}
 \end{array}$$

Chiamiamo questo protocollo modificato DF^* che risulta essere un protocollo ottimo sia per i messaggi sia per il tempo ideale.

Vediamo il codice:

- $S = \{ \text{initiator, idle, available, visited, done} \}$
- $S_{init} = \{ \text{initiator, idle} \}$
- $S_{term} = \{ \text{done} \}$

Initiator:

```

1 spontaneously{
2   initiator = true
3   unvisited = n(x)
4   next = unvisited
5   send T to next
6   send visited to N(x) - next
7   become visited
8 }

```

Idle:

```

1 Receiving(T){
2   unvisited = N(x)
3   first-visit
4 }
5 Receiving(visited){
6   unvisited = N(x) - sender
7   become available
8 }

```

Available:

```

1 Receiving(T){
2     Firs-Visit
3 }
4 Receiving(visited){
5     unvisited = unvisited - sender
6 }

```

Visited:

```

1 Receiving(visited){
2     unvisited = unvisited - sender
3     if(next = sender) then visit
4 }
5
6 Receiving(T) {
7     unvisited = unvisited - sender
8     if(next = sender) then visit
9 }
10
11 Receiving(return){
12     visit
13 }

```

La differenza tra visit e first visit è che si manda a tutti gli altri il messaggio visit, first si esegue quando arriva il token la prima volta, poi si esegue visit mandando il messaggio t a next senza mandare agli altri

First-visit:

```

1 initiator = false
2 unvisited = unvisited - sender
3 if(unvisited != 0) then {
4     next = unvisited
5     send(T) to next
6     send(visited) to N(x) - entry, next
7     become visited
8 } else {
9     send(return) to entry
10    send(visited) to N(x) - entry
11    become Done
12 }

```

Visit:

```

1 if unvisited != 0 then {
2     next = unvisited
3     send(T) to next
4 } else {
5     if(not initiator) then {
6         send(return) to entry
7         become done
8     }
9 }

```

4.4 Spanning Tree

Sarebbe bello poter risolvere i problemi di BW, WP e TR su una rete ad albero, ma non possiamo scegliere la rete. Dovremmo usare una sottorete per minimizzare la complessità di comunicazione, mi conviene usare una sottorete ad albero. Il problema dello spanning tree è data una rete costruire un albero sulla rete. Attenzione ai costi per risolvere i problemi perchè oltre al costo originale del problema eseguito su di un albero dobbiamo aggiungere il costo di costruzione dell'albero.

Ad esempio risolvere BD su un albero costa esattamente $n-1$ messaggi.

Il problema dello spanning tree:

- Costruire una sottorete tale che:
 - coinvolge tutte le entità
 - le entità sono connesse
 - è priva di cicli

La soluzione con un algoritmo distribuito richiede la conoscenza dell'albero all'interno della rete, ogni entità vedrà una piccola parte dell'albero.

Per ogni entità definiamo un $tree_N(x)$ sottoinsieme di $N(x)$, sono tutti i vicini di x che partecipano all'albero, ci deve stare il padre e i figli. Con $Link(Tree_N(x))$ consideriamo gli archi dell'albero, ci sta l'arco x,y se e soltanto se y appartiene a $Tree_N(x)$. L'interno albero Tree è l'unione degli x appartenenti ad e di $Link(Tree_N(x))$.

Risolviamo il problema Spanning Tree con il problema RI (solite restrizioni).

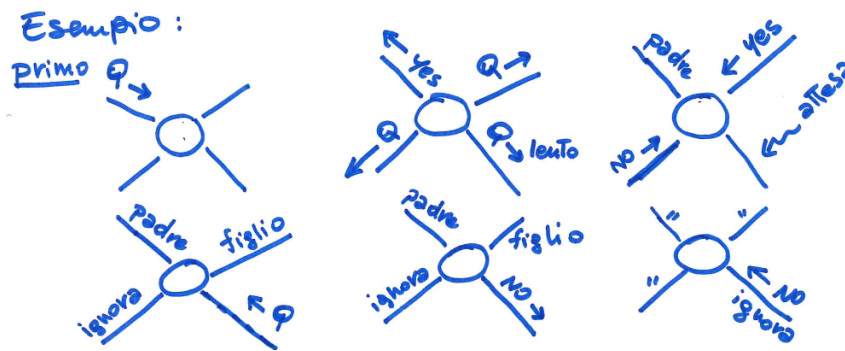
Il protocollo è questo, si chiama SHOUT:

- ogni entità vede solo i suoi $Tree_N(x)$ e tiene traccia del padre
- La radice è data dall'entità che inizia il protocollo
- Strategia di shout: CHIEDI!

Vediamo lo schema:

- Iniziatore $S(\text{root})$ spedisce Q (question) ai suoi vicini e attende le risposte. gli chiede se vogliono diventare i propri figli
- Ogni entità x diversa da s che riceve Q per:
 - La prima volta risponde "yes" e invia Q ai suoi vicini e si mette in attesa
 - una volta successiva alla prima risponde "no"
- Inoltre serve memorizzare l'entità padre e le entità che mi rispondono "yes"
- L'entità termina quando riceve tutte le risposte

Quindi questo protocollo è sostanzialmente un flooding più un replay (per la risposta yes o no).



Vediamo principali idee per la costruzione del codice:

- Si mandano tre tipi di messaggi diversi: Q, yes, no
- ogni entità ha la visibilità locale dell'albero: root, parent, $Tree_N(x)$, counter
- Aggiornare lo stato per raggiungere la terminazione, ricevute le risposte si termina localmente

Ed ecco il protocollo, vediamo prima gli stati:

- $S = \{ \text{iniziatore, inattivo, attivo, finito} \}$
- $S_{init} = \{ \text{iniziatore, inattivo} \}$
- $S_{term} = \{ \text{finito} \}$

Dobbiamo definire le azioni per iniziatore, inattivo e attivo.

Iniziatore:

```

1 impulso spontaneo{
2   root = true
3   counter = 0
4   tree_N(x) != 0
5   send(q) to N(x)
6   become attivo //perche deve ricevere le risposte
7 }

```

Inattivo:

```

1 Ricezione(Q){
2   root = false
3   parent = sender //al primo messaggio q che riceve conosce il padre
4   counter = 1
5   tree_N(x) = {sender}
6   send("yes") sender
7   if counter = |N(x)| then
8     become finito
9   else { //se ha altri vicini deve chiedere agli altri se diventano figli
10    send(q) to N(x) - sender
11    become attivo
12  }
13 }

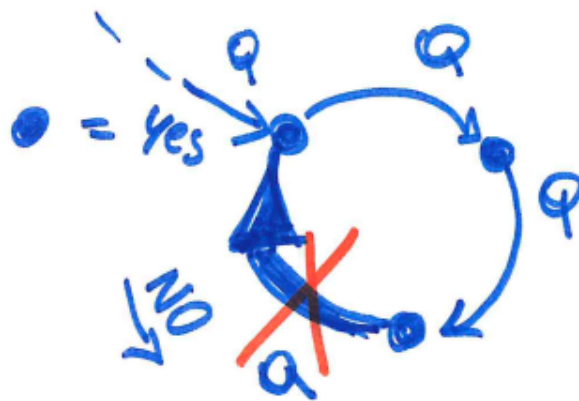
```

Attivo:

```
1 Ricezione(q) {
2   send("NO") to sender
3 }
4
5 Ricezione("yes"){
6   Tree_N(x) = Tree_N(x) u {sender} //vicino vuole essere figlio
7   counter = counter + 1
8   if(counter = |N(x)| then
9     become finito
10 }
11
12 Ricezione("NO"){
13 //non aggiunge il sender tra i figli
14   counter = counter + 1
15   if(counter = |N(x)| then
16     become finito
17 }
```

Vediamo la correttezza:

- Terminazione: in assenza di errori ricevo n risposte pari a Q inviati diventando nello stato finito
- Tutte le entità sono presenti, grazie al flooding di Q. Su ogni arco della rete viaggia un Q, quindi tutte le entità ricevono un Q
- Le entità sono connesse, grazie al fatto che al primo Q rispondo sempre "yes"
- É priva di cicli perchè ogni entità risponde con un solo "yes" tranne la radice che risponde sempre "no"



Questi sono i costi e i lowe bound:

$$M[\text{Shout}] = 2 M[\text{Flooding}] \\ = 2 [2m - (n-1)] = \sim 4m$$

$$T[\text{Shout}] = T[\text{Flooding}] + 1 \\ \leq d + 1$$

Lower Bound

per SPT

$$\begin{aligned} &\rightarrow M[\text{SPT/RI}] \geq m \\ &\rightarrow T[\text{SPT/RI}] \geq d \end{aligned}$$

Possiamo migliorarlo? Creiamo un protocollo shout+ eliminando qualche messaggio per abbassare il 4. Dobbiamo eliminare i messaggi di risposta, i messaggi yes sono necessari e si tengono, quelli no sono superflui e si possono eliminare. Quando ricevo un no è perchè quell'entità è già stata attivata da qualche nodo, quindi significa che riceveremo da quella entità un messaggio Q, potremmo interpretare quel messaggio Q come una risposta no evitando di rispondere no. Così eliminiamo tutti i messaggi no.

Allora su ogni arco viaggia un q richiesta e una risposta yes o q, su ogni arco viaggiano 2 messaggi, quindi il numero di messaggi è 2m.

Si può far notare che un'altra soluzione per lo Spanning Tree è utilizzare il protocollo Traversal per costruire sequenzialmente l'albero, in questo caso Tree sarebbe l'insieme di archi che sono l'insieme dei link su cui viaggiano i messaggi "return".

4.5 Election

Questo problema prevede un insieme di entità autonome ma che non si possono distinguere dall'esterno, si vuole eleggere un'entità in particolare affinché diventi il capo delle altre, quella eletta deve diventare leader e quelle che lo seguono follower.

Ci sono applicazioni complesse in cui c'è bisogno di un entità centrale che coordini le altre.

Abbiamo un risultato di impossibilità, se le entità sono omogenee e all'occhio esterno sono identiche è impossibile deterministicamente determinare un leader sotto le restrizioni R. Possiamo dare un'idea del perchè succede questa cosa, supponiamo di avere due entità x e y omogenee (contenuti registri uguali), eseguono entrambe lo stesso algoritmo e si ritrovano in perfetta simmetria, quindi si ritrovano dopo aver eseguito un'azione in perfetta simmetria.

C'è un risultato di possibilità abbastanza ovvio, sotto restrizioni RI, la starting entità diventa immediatamente LEADER. Però chi stabilisce chi è l'entità leader? lo stabilisce un evento esterno al sistema, l'approccio diventa banale, perchè da fuori decidiamo chi

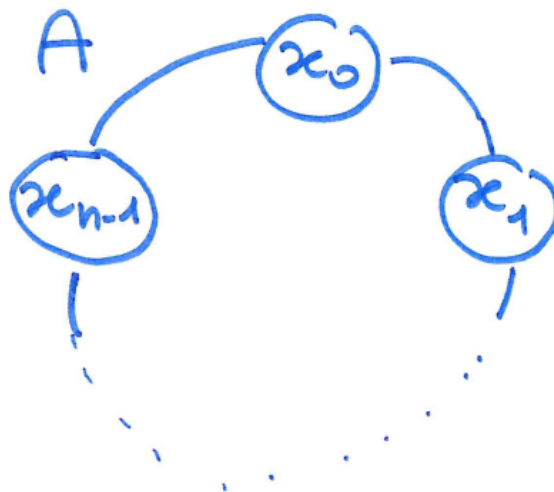
diventa il leader, non sono le entità che scelgono autonomamente chi deve essere il leader. Quindi toglieremo questa restrizione e ne metteremo un'altra.

La nuova restrizione si chiama Initial Distinct Values (ID), è una restrizione in cui ogni entità ha il proprio nome $id(x)$ è il nome di x nell'ambiente distribuito (o valore di x). Quando usiamo le restrizioni R e le ID le denotiamo tutte con IR .

Le strategie di soluzione giocano sul fatto che le entità hanno un nome, quindi perché non focalizzarci sui nomi delle entità e si riesce a eleggere un numero particolare come il minimo o il massimo:

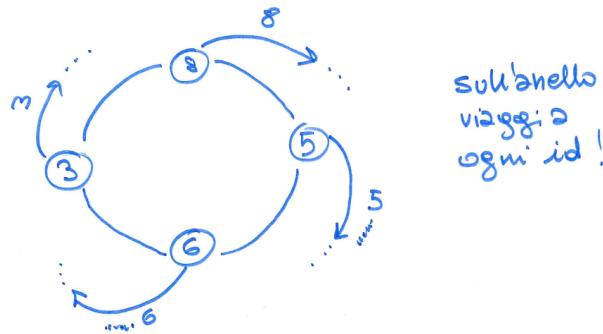
- Elect Minimum
 - Trova $id(x)$ minimo e fai diventare x LEADER
 - Tutte le altre entità che non sono x diventano FOLLOWER
- Elect Minimum Initiator
 - Trova $id(x)$ minimo solo tra quelle che hanno iniziato il protocollo, quando lo hai individuato diventa LEADER
 - Tutte le altre entità diventano FOLLOWER

Risolviamo il problema Elect Minimum in una topologia di rete particolare, ad anello. Le entità sono collegate ad anello. Dobbiamo aggiungere una restrizione: ogni entità x sa di essere in una rete che è un anello. Siccome hanno solo due vicini, quando ricevono un messaggio da un vicino l'altro lo chiamiamo other, other è quello a cui devono inoltrare il messaggio che hanno ricevuto dall'altro vicino. Un'altra cosa da evidenziare abbiamo che vale $m = n$ (numero di link = numero di entità).



Una possibile soluzione in questa situazione è il protocollo ALL THE WAY, i nostri messaggi vengono inoltrati nell'anello per un certo periodo di tempo, le entità inoltrano i messaggi nella stessa direzione. Le entità ogni volta che ricevono un messaggio lo inoltrano fino a che il messaggio viene tolto quando ha fatto il giro dell'anello. Il messaggio ha due campi: sigla "Elect" e il mittente $id(x)$.

y genera un messaggio Elect e lo inoltra a x, la quale inoltra il messaggio, se è la prima volta che è stata svegliata dal protocollo inoltra anche un' altro messaggio di Elect in cui mette il suo id(x). Osserviamo che ogni entità x vede id(y), questi messaggi che girano tra le entità portano alla conoscenza di tutte le entità tutte le altre entità nell'anello. Quindi tutte le entità possono calcolarsi il valore minimo di id(x) nell'anello.



Quando ritorna ad un'entità il proprio messaggio sa che tutti hanno il suo id e significa che il messaggio ha fatto il giro della rete e decide di non inoltrarlo più. Ma a questo punto l'entità può terminare? Se rispondestimo di sì supporremmo che sia nella rete la restrizione Message Ordering, ovvero che il prelievo dei messaggi sui link segue una politica FIFO, ma noi non abbiamo questa restrizione! Se ha visto n messaggi comunque non può terminare, perché staremmo supponendo che ogni entità conosce il numero di entità nell'anello. Quindi non può terminare, dobbiamo fare in modo che quando costruisce il messaggio nell'anello ci mette un contatore che ogni entità nell'anello aggiorna con un +1, così facendo quando gli ritorna il messaggio sa quante entità ci sono nell'anello. Allora il messaggio assume questa forma: $E = ("Elect", id(x), counter)$. Come usiamo counter:

- Inizialmente l'entità x pone counter = 1
- Ogni altra entità y che inoltra E somma 1 a counter
- Quando E torna all'entità x counter è uguale a $n = |A|$
- Se x ha già ricevuto n diversi id può decidere di terminare, altrimenti aspetta di ricevere quelli che ancora non sono arrivati a lui, inoltre deve continuare l'operazione di inoltro dei messaggi

Il codice è questo, partiamo dagli stati:

- Stati = {Asleep, awake, leader, follower}
- StatiInit = {Asleep}
- StatiFinal{leader, follower}

Asleep:

```

1 spontaneously {
2   INITIALIZE
3   become awake
4 }

```

```

5
6 Receiving(" Elect", value, counter){
7     INITIALIZE
8     send(" Elect", value, counter + 1) to OTHER
9     min = Min{min, value} //si tiene in memoria l'id minimo per LEADER
10    count = count + 1
11    become awake
12 }

```

INITIALIZE:

```

1 count = 0
2 size = 1 //size dell'anello
3 know = false //true quando entita riceve proprio messaggio
4 send(" Elect", value, size) to right
5 min = id(x) //setta LEADER a se stesso

```

Awake:

```

1 Receiving(" Elect", value, counter) {
2     if value != id(x) then { //messaggio proprio o da inoltrare
3         send(" Elect", value, counter + 1) to OTHER
4         min = Min{min, value} //si tiene in memoria l'id minimo per LEADER
5         count = count + 1
6         if know = true then CHECK //controllo se devo terminare
7     } else {
8         //ricevo il mio messaggio
9         size = counter //so il numero n di entita nell'anello
10        know = true
11        CHECK
12    }
13 }

```

```

1 Procedure CHECK { //invcata quando sa n entita in anello
2     if count = size then {
3         if min = id(x) then {
4             become leader
5         } else {
6             become follower
7         }
8     }
9 }

```

Vediamo la complessità, la quantità di messaggio all the way unito alle restrizioni IR e che sappiamo di essere in un anello è n^2 perchè ogni messaggio generato da un entità richiede n messaggi. Troppo costoso!

Proviamo la strategia Elect Minimum Initiator in cui solo gli initiator generano E, mentre le altre entità inoltrano i messaggi. Abbiamo un problema di terminazione, sebbene ogni entità initiator scopre quante entità ci sono nell'anello le altre non lo sanno. Sono gli initiator che quando hanno capito chi è il leader devono avvisare gli altri. In particolare è il leader che invia un messaggio nell'anello di fine del protocollo, n messaggi di fine.

Supponiamo che k sia il numero di initiator, si generano k messaggi che producono n messaggi quindi in totale abbiamo $n \cdot k + n$. Il tempo invece non è uguale ai messaggi, perchè

molti di questi viaggiano in parallelo, quindi per scoprire il tempo bisogna considerare il caso peggiore, sono due le entità che si attivano e sono a distanza n . C'è un primo tempo n per l'invio di un messaggio che fa tutto il ciclo, poi c'è l'invio di un altro messaggio e poi gli n messaggi di fine protocollo sequenziali. Otteniamo quindi un tempo $3n - 1$. Si può raggiungere con due initiator che si svegliano in momenti diversi. Lasciando perdere le costanti un messaggio quando viene generato deve fare il giro intorno all'anello quindi abbiamo un tempo $O(n)$.

4.6 Routing

Ci sono due entità che devono comunicare, x vuole inviare un messaggio a y e deve avere indicazioni del cammino sul grafo G . È vero che non c'è un solo cammino nel grafo, nel routing va bene un cammino qualunque, vedremo poi lo Shortest Path in cui dovremo individuare il cammino migliore (cammino più corto in un grafo non pesato o il cammino di costo minimo se il grafo è pesato). Le applicazioni sono proprio quelle della comunicazione.

Per risolvere il problema del routing potrebbe essere sufficiente rifarsi ad un problema che abbiamo già visto, il problema BROADCAST, perchè se x vuole mandare un messaggio a y se lo manda a tutte le entità della rete sicuramente lo manda anche a y . Questa soluzione è fortemente inefficiente.

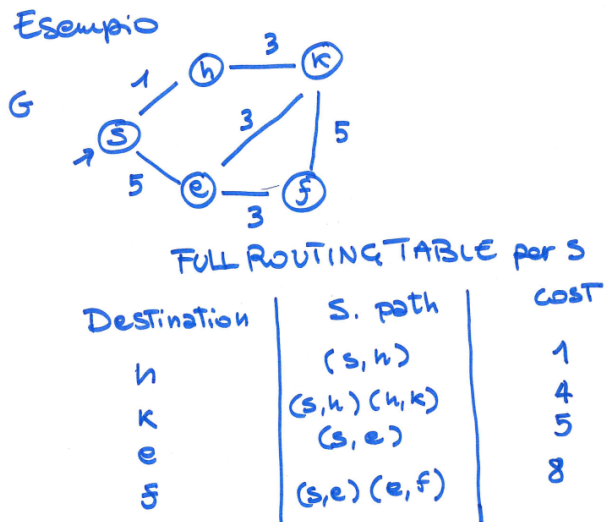
Scegliamo un cammino ben definito a caso tra x e y . Se invece vogliamo abbassare i costi di comunicazione dovremmo individuare il cammino migliore. Per fare questo richiedo l'uso della memoria per registrare informazioni sui costi di G per ogni entità al fine di calcolare i cammini minimi verso ogni altra entità.

4.6.1 Shortest Path

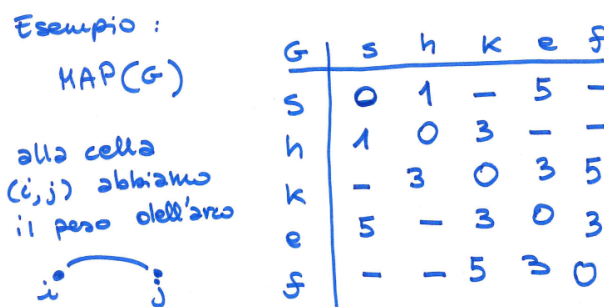
Dobbiamo mettere delle restrizioni, per questo problema pensiamo alle restrizioni IR (abbiamo l'identification value per le entità, un etichetta numerica di x). Le strategie per risolvere questo problema differiscono dal tipo di informazioni che le entità si tengono in memoria, perchè posso costruire una tabella interna al nodo del grafo e scegliere che attributi memorizzare.

Partiamo definendo la FULL ROUTING TABLE perchè ogni strategia alla fine ha bisogno di questa tabella per risolvere il problema. Immaginiamo il grafo G , ogni link ha un costo per poter essere attraversato. La FULL ROUTING TABLE per l'entità S è così composta:

- Destinazione: tutte le altre entità della rete
- S.Path: si calcola lo shortest path
- Cost: il costo totale di ogni shortest path



Il primo protocollo che vediamo è il GOSSIPING, prima di costruire la FULL ROUTING TABLE ha bisogno di tutte le entità id e di ogni arco il peso, allora si costruisce la mappa di G, che altro non è una matrice di adiacenza del grafo (i numeri sono il peso dei link diretti) e poi all'occorrenza a partire dalla matrice si calcola le righe della Full Routing Table:



Per costruire Map(G) è sufficiente che ogni entità x diffonda le proprie informazioni sui vicini ad ogni altra entità y. x fa un broadcast dei suoi vicini e dei costi. Visto che broadcast su tutta la rete è costoso si può pensare di fare questa diffusione utilizzando un albero riducendo di molto il costo dei messaggi. Map Gossip:

- Costruzione di un albero T per G, utilizzato per far passare i messaggi
- Scambio di informazioni con i vicini, si scambiano id e costi dei link
- ogni entità diffonde le sue informazioni a tutte le altre entità usando i link di T

Vediamo la complessità, il numero di messaggi scambiati sono così composti:

- Costo costruzione spanning Tree T: $= (m + n \log n)$ che è il miglior costo possibile
- Acquisizione di info dei vicini: $2m$ (ogni vicino spedisce id e costo del link)
- broadcast di info su T: $2m \cdot n$ i $2m$ messaggi devono attraversare tutti i link dell'albero

Facendo la somma di questi costi otteniamo una quantità di messaggi $2m \cdot n$ se il grafo è sparso otteniamo circa $O(n^2)$.

Il tempo è difficile da calcolare, entra in gioco il diametro dell'albero. Questo protocollo viene scartato perchè tenere la mappa del grafo è molto oneroso in termini di memoria.

Spieghiamo quindi il protocollo Iterated-Construction in cui ogni entità si costruisce la sua FULL ROUTING TABLE a più riprese evitando così lo spreco di memoria (nell'ordine di $n \times n$).

Si parte con una FRT che è parzialmente completata, contiene solo le informazioni dei vicini:

Esempio: F.R.T. iniziale di s:

Destination	S. Path	Cost
h	(s,h)	1
k	-	∞
e	(s,e)	5
f	-	∞

costo infinito che verrà sostituito alle iterazioni succes.

Notiamo che la FRT di un entità x non necessariamente deve contenere l'interno shortest path per raggiungere una qualunque altra entità z. È sufficiente tenere traccia del vicino coinvolto nello shortest path per z.

Le informazioni di cui vogliamo tenere traccia per raggiungere ad esempio l'entità z sono:

- Costo dello s.p. per z
- il primo link dello s.p. che si traduce in un nodo

Per aggiornare la Full Routing Table dobbiamo definire il Distance Vector, è una full routing table parziale ristretta alle colonne destinazione e costo e viene indicata con V. Usiamo questa notazione $V_x[z]$ è il costo del cammino minimo da x a z.

L'iterazione funziona così:

- Ogni entità diffonde la propria V ai suoi vicini
- sulla base delle informazioni che gli arrivano dai vicini stabilisce se sono stati trovati cammini minimi migliori di quelli che ha nella propria F.R.T. e in tal caso aggiorna la F.R.T.

Si può dimostrare che con n-1 iterazioni si costruisce correttamente la F.R.T.

Come fa un entità x ad individuare ad ogni iterazione il cammino minimo per raggiungere una certa entità z? Sia i un iterazione e v_y il vettore distance vector all'iterazione i, indicizzato a z è il costo del cammino minimo da y a z ottenuto da y alla i-esima iterazione, questo vettore v_y alla i+1-esima iterazione questo costo arriva ai vicini di y. Così x calcola alla i+1-esima iterazione ha a disposizione il vettore v_y e cerca di capire se il suo cammino minimo per z è peggiore del cammino che hanno individuato le altre entità e fa questo conto: $w[z] = \min_{y \in N(x)} \{o(x,y) + V_y^i[z]\}$ dove $o(x,y)$ è il costo del link (x, y). Se $w[z] < V_x^i[z]$ x sceglie come costo per lo s.p. per z aggiornando la sua F.R.T. e memorizza anche sotto la colonna shortest path il vicino y che mi ha dato il costo minimo.

Il vantaggio di questo approccio è la memoria, la F.R.T. richiede meno spazio di Map(G). Esempio di F.R.T. finale per l'entità S:

Esempio di F.R.T. finale per l'entità S

Destination	s. path.	cost
h	h	1
k	h	4
e	e	5
f	e	8

Vediamo la complessità, teniamo conto che è un protocollo con iterazioni e il numero di iterazioni è $n-1$.

→ n messaggi

$$M[\text{It-constr. / IR}] = \underbrace{2m \cdot n \cdot (n-1)}_{\substack{\downarrow \\ \text{\# di msg per spedire } V}}$$

→ Tempo

$$T[\text{It-constr. / IR}] = (n-1) \cdot \underbrace{\mathcal{C}(n)}_{\substack{\downarrow \\ \text{Tempo ideale per trasm. } V}}$$

$$\mathcal{C}(n) = \begin{cases} O(1) & \text{costante *} \\ & \text{quando q consente} \\ & \text{msg lunghi} \\ O(n) & \end{cases}$$

CASO *: T è lineare in n e $M = O(m \times n)$